just lock acquisition. In such a situation, multi-granularity locking can be used to avoid multiple requests. For example, if multiple data items are stored in a page, a single page lock (which is at a coarser granularity) can avoid the need to acquire multiple data-item locks (which are at a finer granularity). This strategy works well if there is very little lock contention, but with higher contention, acquiring a coarse granularity lock can affect concurrency significantly.

Lock de-escalation, is a way of adaptively decreasing the lock granularity if there is higher contention. Lock de-escalation is initiated by the data server sending a request to the client to de-escalate a lock, and the client responds by acquiring finer-granularity locks and then releasing the coarser-granularity lock.

When switching to a finer granularity, if some of the locks were for cached data items that are not currently locked by any transaction at a client, the data item can be removed from the cache instead of acquiring a finer-granularity lock on it.

20.4 Parallel Systems

Parallel systems improve processing and I/O speeds by using a large number of computers in parallel. Parallel machines are becoming increasingly common, making the study of parallel database systems correspondingly more important.

In parallel processing, many operations are performed simultaneously, as opposed to serial processing, in which the computational steps are performed sequentially. A **coarse-grain** parallel machine consists of a small number of powerful processors; a **massively parallel** or **fine-grain parallel** machine uses thousands of smaller processors. Virtually all high-end server machines today offer some degree of coarse-grain parallelism, with up to two or four processors each of which may have 20 to 40 cores.

Massively parallel computers can be distinguished from the coarse-grain parallel machines by the much larger degree of parallelism that they support. It is not practical to share memory between a large number of processors. As a result, massively parallel computers are typically built using a large number of computers, each of which has its own memory, and often, its own set of disks. Each such computer is referred to as a **node** in the system.

Parallel systems at the scale of hundreds to thousands of nodes or more are housed in a **data center**, which is a facility that houses a large number of servers. Data centers provide high-speed network connectivity within the data center, as well as to the outside world. The numbers and sizes of data centers have grown tremendously in the last decade, and modern data centers may have several hundred thousand servers.

20.4.1 Motivation for Parallel Databases

The transaction requirements of organizations have grown with the increasing use of computers. Moreover, the growth of the World Wide Web has created many sites with millions of viewers, and the increasing amounts of data collected from these viewers has produced extremely large databases at many companies.

The driving force behind parallel database systems is the demands of applications that have to query extremely large databases (of the order of petabytes—that is, 1000 terabytes, or equivalently, 10^{15} bytes) or that have to process an extremely large number of transactions per second (of the order of thousands of transactions per second). Centralized and client–server database systems are not powerful enough to handle such applications.

Web-scale applications today often require hundreds to thousands of nodes (and in some cases, tens of thousands of nodes) to handle the vast number of users on the web.

Organizations are using these increasingly large volumes of data—such as data about what items people buy, what web links users click on, and when people make telephone calls—to plan their activities and pricing. Queries used for such purposes are called **decision-support queries**, and the data requirements for such queries may run into terabytes. Single-node systems are not capable of handling such large volumes of data at the required rates.

The set-oriented nature of database queries naturally lends itself to parallelization. A number of commercial and research systems have demonstrated the power and scalability of parallel query processing.

As the cost of computing systems has reduced significantly over the years, parallel machines have become common and relatively inexpensive. Individual computers have themselves become parallel machines using multicore architectures. Parallel databases are thus quite affordable even for small organizations.

Parallel database systems which can support hundreds of nodes have been available commercially for several decades, but the number of such products has seen a significant increase since the mid 2000s. Open-source platforms for parallel data storage such as the Hadoop File System (HDFS), and HBase, and for query processing, such as Hadoop Map-Reduce and Hive (among many others), have also seen extensive adoption.

It is worth noting that application programs are typically built such that they can be executed in parallel on a number of application servers, which communicate over a network with a database server, which may itself be a parallel system. The parallel architectures described in this section can be used not only for data storage and query processing in the database but also for parallel processing of application programs.

20.4.2 Measures of Performance for Parallel Systems

There are two main measures of performance of a database system: (1) **throughput**, the number of tasks that can be completed in a given time interval, and (2) **response time**, the amount of time it takes to complete a single task from the time it is submitted. A system that processes a large number of small transactions can improve throughput by processing many transactions in parallel. A system that processes large transactions can improve response time as well as throughput by performing subtasks of each transaction in parallel.

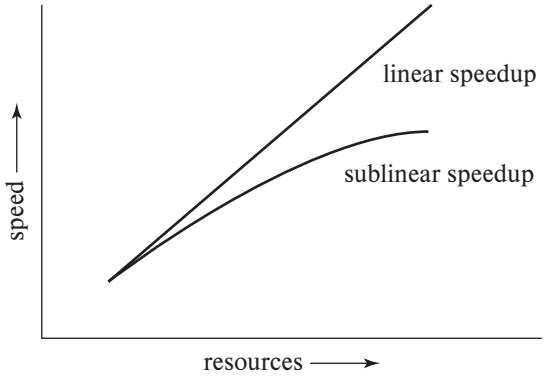

Figure 20.2 Speedup with increasing resources.

Parallel processing within a computer system allows database-system activities to be speeded up, allowing faster response to transactions, as well as more transactions to be executed per second. Queries can be processed in a way that exploits the parallelism offered by the underlying computer system.

Two important issues in studying parallelism are speedup and scaleup. Running a given task in less time by increasing the degree of parallelism is called **speedup**. Handling larger tasks by increasing the degree of parallelism is called **scaleup**.

Consider a database application running on a parallel system with a certain number of processors and disks. Now suppose that we increase the size of the system by increasing the number of processors, disks, and other components of the system. The goal is to process the task in time inversely proportional to the number of processors and disks allocated. Suppose that the execution time of a task on the larger machine is T_L, and that the execution time of the same task on the smaller machine is T_S. The speedup due to parallelism is defined as T_S/T_L. The parallel system is said to demonstrate **linear speedup** if the speedup is N when the larger system has N times the resources (processors, disk, and so on) of the smaller system. If the speedup is less than N, the system is said to demonstrate **sublinear speedup**. Figure 20.2 illustrates linear and sublinear speedup.[2]

Scaleup relates to the ability to process larger tasks in the same amount of time by providing more resources. Let Q be a task, and let Q_N be a task that is N times bigger than Q. Suppose that the execution time of task Q on a given machine M_S is T_S, and the execution time of task Q_N on a parallel machine M_L that is N times larger than M_S is T_L. The scaleup is then defined as T_S/T_L. The parallel system M_L is said to demonstrate **linear scaleup** on task Q if $T_L = T_S$. If $T_L > T_S$, the system is said

[2]In some cases, a parallel system may provide superlinear speedup, that is, an N times larger system may provide speedup greater than N. This could happen, for example, because data that did not fit in the main memory of a smaller system do fit in the main memory of a larger system, avoiding disk I/O. Similarly, data may fit in the cache of a larger system, reducing memory accesses compared to a smaller system, which could lead to superlinear speedup.

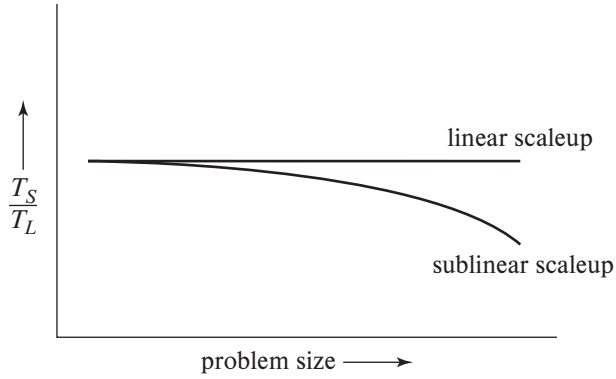

Figure 20.3 Scaleup with increasing problem size and resources.

to demonstrate **sublinear scaleup**. Figure 20.3 illustrates linear and sublinear scaleups (where the resources increase in proportion to problem size). There are two kinds of scaleup that are relevant in parallel database systems, depending on how the size of the task is measured:

- In **batch scaleup**, the size of the database increases, and the tasks are large jobs whose runtime depends on the size of the database. An example of such a task is a scan of a relation whose size is proportional to the size of the database. Thus, the size of the database is the measure of the size of the problem. Batch scaleup also applies in scientific applications, such as executing a weather simulation at an N-times finer resolution,[3] or performing the simulation for an N-times longer period of time.
- In **transaction scaleup**, the rate at which transactions are submitted to the database increases, and the size of the database increases proportionally to the transaction rate. This kind of scaleup is what is relevant in transaction-processing systems where the transactions are small updates—for example, a deposit or withdrawal from an account—and transaction rates grow as more accounts are created. Such transaction processing is especially well adapted for parallel execution, since transactions can run concurrently and independently on separate nodes, and each transaction takes roughly the same amount of time, even if the database grows.

Scaleup is usually the more important metric for measuring the efficiency of parallel database systems. The goal of parallelism in database systems is usually to make sure that the database system can continue to perform at an acceptable speed, even as the size of the database and the number of transactions increases. Increasing the ca-

[3]For example, a weather simulation that divides the atmosphere in a particular region into cubes of side 200 meters may need to be modified to use a finer resolution, with cubes of side 100 meters; the number of cubes would thus be scaled up by a factor of 8.

pacity of the system by increasing the parallelism provides a smoother path for growth for an enterprise than does replacing a centralized system with a faster machine (even assuming that such a machine exists). However, we must also look at absolute performance numbers when using scaleup measures; a machine that scales up linearly may perform worse than a machine that scales less than linearly, simply because the latter machine is much faster to start off with.

A number of factors work against efficient parallel operation and can diminish both speedup and scaleup.

- **Sequential computation**. Many tasks have some components that can benefit from parallel processing, and some components that have to be executed sequentially. Consider a task that takes time T to run sequentially. Suppose the fraction of the total execution time that can benefit from parallelization is p, and that part is executed by n nodes in parallel. Then the total time taken would be $(1-p)T+(p/n)T$, and the speedup would be $\frac{1}{(1-p)+(p/n)}$. (This formula is referred to as **Amdahl's law**.) If the fraction p is, say $\frac{9}{10}$, then the maximum speedup possible, even with very large n, would be 10.

  Now consider scaleup, where the problem size increases. If the time taken by the sequential part of a task increases along with the problem size, scaleup will be similarly limited. Suppose fraction p of the execution time of a problem benefits from increasing resources, while fraction $(1-p)$ is sequential and does not benefit from increasing resources. Then the scaleup with n times more resources on a problem that is n times larger will be $\frac{1}{n(1-p)+p}$. (This formula is referred to as **Gustafson's law**.) However, if the time taken by the sequential part does not increase with problem size, its impact on scaleup will be less as the problem sizes.

  Start-up costs. There is a start-up cost associated with initiating a single process. In a parallel operation consisting of thousands of processes, the *start-up time* may overshadow the actual processing time, affecting speedup adversely.

- **Interference**. Since processes executing in a parallel system often access shared resources, a slowdown may result from the *interference* of each new process as it competes with existing processes for commonly held resources, such as a system bus, or shared disks, or even locks. Both speedup and scaleup are affected by this phenomenon.

- **Skew**. By breaking down a single task into a number of parallel steps, we reduce the size of the average step. Nonetheless, the service time for the single slowest step will determine the service time for the task as a whole. It is often difficult to divide a task into exactly equal-sized parts, and the way that the sizes are distributed is therefore *skewed*. For example, if a task of size 100 is divided into 10 parts, and the division is skewed, there may be some tasks of size less than 10 and some tasks of size more than 10; if even one task happens to be of size 20, the speedup obtained by running the tasks in parallel is only 5, instead of 10 as we would have hoped.

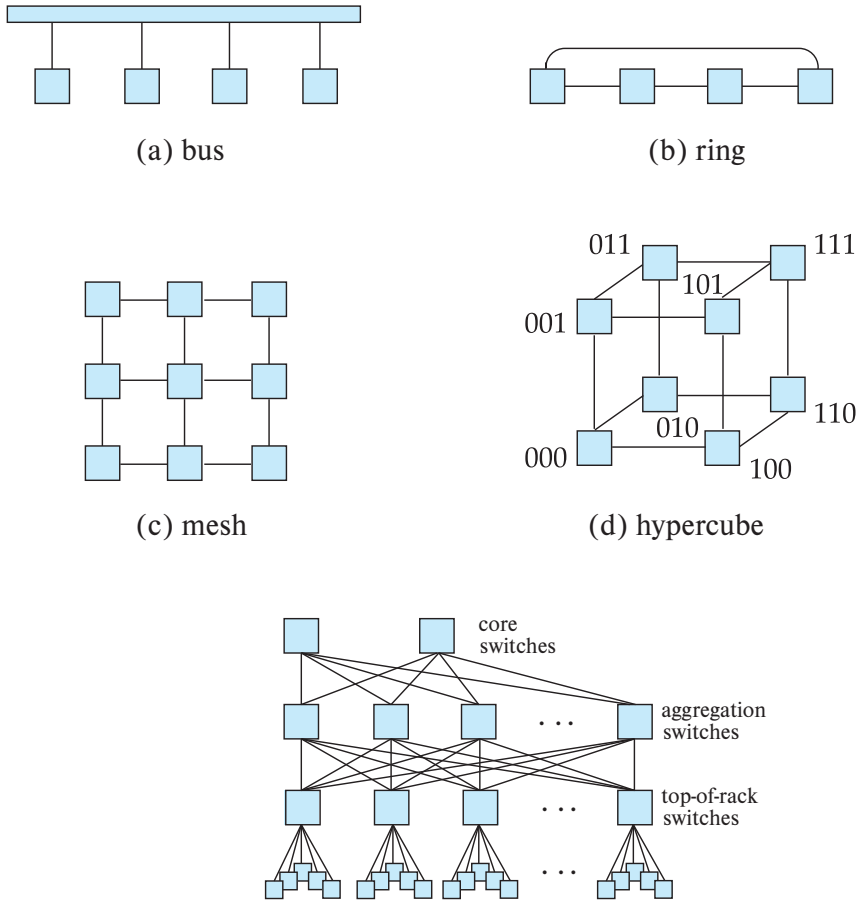

Figure 20.4 Interconnection networks.

20.4.3 Interconnection Networks

Parallel systems consist of a set of components (processors, memory, and disks) that can communicate with each other via an **interconnection network**. Figure 20.4 shows several commonly used types of interconnection networks:

- **Bus**. All the system components can send data on and receive data from a single communication bus. This type of interconnection is shown in Figure 20.4a. Bus interconnects were used in earlier days to connect multiple nodes in a network, but they are no longer used for this task. However, bus interconnections are still used for connecting multiple CPUs and memory units within a single node, and they work well for small numbers of processors. However, they do not scale well

with increasing parallelism, since the bus can handle communication from only one component at a time; with increasing numbers of CPUs and memory banks in a node, other interconnection mechanisms such as ring or mesh interconnections are now used even within a single node.

- **Ring**. The components are nodes arranged in a ring (circle), and each node is connected to its two adjacent nodes in the ring, as shown in Figure 20.4b. Unlike a bus, each link can transmit data concurrently with other links in the ring, leading to better scalability. However, to transmit data from one node to another node on the ring may require a large number of hops; specifically, up to $n/2$ hops may be needed on a ring with n nodes, assuming communication can be done in either direction on the ring. Furthermore, the transmission delay increases if the number of nodes in the ring is increased.

- **Mesh**. The components are nodes in a grid, and each component connects to all its adjacent components in the grid. In a two-dimensional mesh, each node connects to (up to) four adjacent nodes, while in a three-dimensional mesh, each node connects to (up to) six adjacent nodes. Figure 20.4c shows a two-dimensional mesh. Nodes that are not directly connected can communicate with one another by routing messages via a sequence of intermediate nodes that are directly connected to one another. The number of communication links grows as the number of components grows, and the communication capacity of a mesh therefore scales better with increasing parallelism.

  Mesh interconnects are used to connect multiple cores in a processor, or processors in a single server, to each other; each processor core has direct access to a bank of memory connected to the processor core, but the system transparently fetches data from other memory banks by sending messages over the mesh interconnects.

  However, mesh interconnects are no longer used for interconnecting nodes, since the number of hops required to transmit data increases significantly with the number of nodes (the number of hops required to transmit data from one node to another node in a mesh is proportional in the worst case to the square root of the number of nodes). Parallel systems today have very large numbers of nodes, and mesh interconnects would thus be impractically slow.

- **Hypercube**. The components are numbered in binary, and a component is connected to another if the binary representations of their numbers differ in exactly one bit. Thus, each of the n components is connected to $\log(n)$ other components. Figure 20.4d shows a hypercube with eight nodes. In a hypercube interconnection, a message from a component can reach any other component by going through at most $\log(n)$ links. In contrast, in a mesh architecture a component may be $2(\sqrt{n} - 1)$ links away from some of the other components (or $\sqrt{n}$ links away, if the mesh interconnection wraps around at the edges of the grid). Thus communication delays in a hypercube are significantly lower than in a mesh.

Hypercubes have been used to interconnect nodes in massively parallel computers in earlier days, but they are no longer commonly used.

- **Tree-like**. Server systems in a data center are typically mounted in racks, with each rack holding up to about 40 nodes. Multiple racks are used to build systems with larger numbers of nodes. A key issue is how to interconnect such nodes.

  To connect nodes within a rack, there is typically a network switch mounted at the top of the rack; 48 port switches are commonly used, so a single switch can be used to connect all the servers in a rack. Current-generation network switches typically support a bandwidth of 1 to 10 gigabits per second (Gbps) simultaneously from/to each of the servers connected to the switch, although more expensive network interconnects with 40 to 100 Gbps bandwidths are available.

  Multiple top-of-rack switches (also referred to as **edge switches**) can in turn be connected to another switch, called an **aggregation switch**, allowing interconnection between racks. If there are a large number of racks, the racks may be divided into groups, with one aggregation switch connecting a group of racks, and all the aggregation switches in turn connected to a **core switch**. Such an architecture is a **tree topology** with three tiers. The core switch at the top of the tree also provides connectivity to outside networks.

  A problem with this basic tree structure, which is frequently used in local-area networks within organizations, is that the available bandwidth between racks is often not sufficient if multiple machines in a rack try to communicate significant amounts of data with machines from other racks. Typically, the interconnects of the aggregation switches support higher bandwidths of 10 to 40 Gbps, although interconnects of 100 Gbps are available. Interconnects of even higher capacity can be created by using multiple interconnects in parallel. However, even such high-speed links can be saturated if a large enough number of servers in a rack attempt to communicate at their full connection bandwidth to servers in other racks.

  To avoid the bandwidth bottleneck of a tree structure, data centers typically connect each top-of-rack (edge) switch to multiple aggregation switches. Each aggregation switch in turn is linked to a number of core switches at the next layer. Such an interconnection topology is called a **tree-like topology**; Figure 20.4e shows a tree-like topology with three tiers. The tree-like topology is also referred to as a **fat-tree topology**, although originally the fat-tree topology referred to a tree topology where edges higher in the tree have a higher bandwidth than edges lower in the tree.

  The benefit of the tree-like architecture is that each top-of-rack switch can route its messages through any of the aggregation switches that it is connected to, increasing the inter-rack bandwidth greatly as compared to the tree topology. Similarly, each aggregation switch can communicate with another aggregation switch via any of the core switches that it is connected to, increasing the bandwidth available between the aggregation switches. Further, even if an aggregation or edge switch fails, there are alternative paths through other switches. With appropriate

routing algorithms, the network can continue functioning even if a switch fails, making the network *fault-tolerant*, at least to failures of one or a few switches.

A tree-like architecture with three tiers can handle a **cluster** of tens of thousands of machines. Although a tree-like topology improves the inter-rack bandwidth greatly compared to a tree topology, parallel processing applications, including parallel storage and parallel database systems, perform best if they are designed in a way that reduces inter-rack traffic.

The tree-like topology and variants of it are widely used in data centers today. The complex interconnection networks in a data center are referred to as a **data center fabric**.

While network topologies are very important for scalability, a key to network performance is network technology used for individual links. The popular technologies include:

- **Ethernet**: The dominant technology for network connections today is the Ethernet technology. Ethernet standards have evolved over time, and the predominant versions used today are 1-gigabit Ethernet and 10-gigabit Ethernet, which support bandwidths of 1 and 10 gigabits per second respectively. Forty-gigabit Ethernet and 100-gigabit Ethernet technologies are also available at a higher cost and are seeing increasing usage. Ethernet protocols can be used over cheaper copper cables for short distances, and over optical fiber for longer distances.
- **Fiber channel**: The Fiber Channel Protocol standard was designed for high-speed interconnection between storage systems and computers, and it is predominantly used to implement storage area networks (described in Section 20.4.6). The different versions of the standard have supported increasing bandwidth over the years, with 16 gigabits per second available as of 2011, and 32 and 128 gigabits per second supported from 2016.
- **Infiniband**: The Infiniband standard was designed for interconnections with a data center; it was specifically designed for high-performance computing applications which need not just very high bandwidth, but also very low latency. The Infiniband standard has evolved, with link speeds of 8-gigabits per second available by 2007 and 24-gigabits per second available by 2014. Multiple links can be aggregated to give a bandwidth of 120 to 290-gigabits per second.

  The latency associated with message delivery is as important as bandwidth for many applications. A key benefit of Infiniband is that it supports latencies as low as 0.7 to 0.5 microseconds. In contrast, Ethernet latencies can be up to hundreds of microseconds in an unoptimized local-area network, while latency-optimized Ethernet implementations still have latencies of several microseconds.

One of the important techniques used to reduce latency is to allow applications to send and receive messages by directly interfacing with the hardware, bypassing the operating system. With the standard implementations of the networking stack, applications

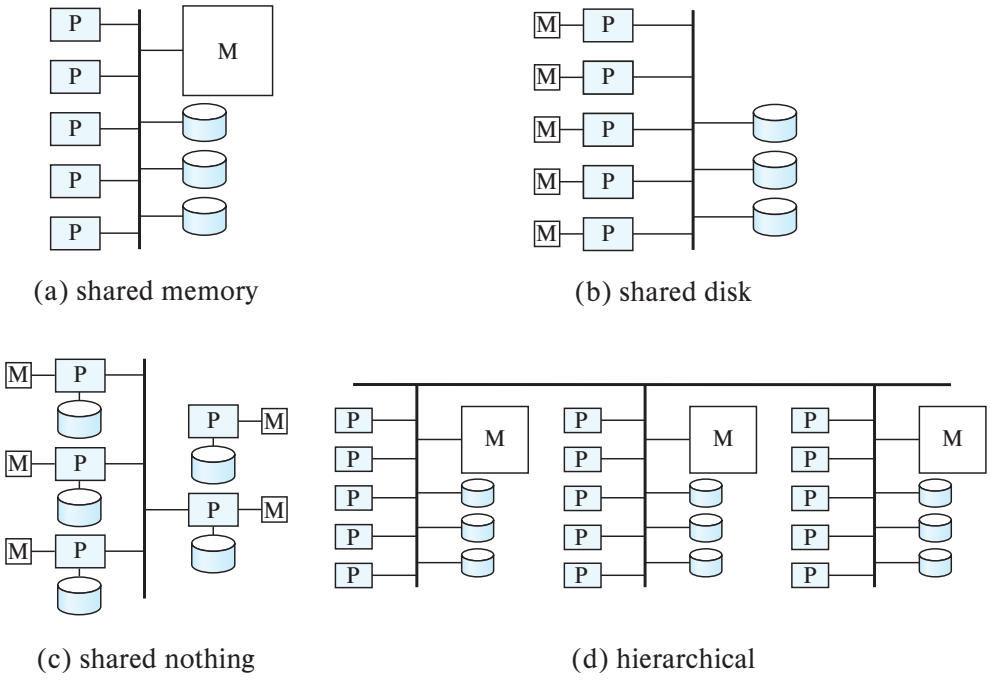

Figure 20.5 Parallel database architectures.

send messages to the operating system, which in turn interfaces with the hardware, which in turn delivers the message to the other computer, where again the hardware interfaces with the operating system, which then interfaces with the application to deliver the message. Support for direct access to the network interface, bypassing the operating system, reduces the communication latency significantly.

Another approach to reducing latency is to use **remote direct memory access** (**RDMA**), a technology which allows a process on one node to directly read or write to memory on another node, without explicit message passing. Hardware support ensures that RDMA can transfer data at very high rates with very low latency. RDMA implementations can use Infiniband, Ethernet, or other networking technologies for physical communication between nodes.

20.4.4 Parallel Database Architectures

There are several architectural models for parallel machines. Among the most prominent ones are those in Figure 20.5 (in the figure, M denotes memory, P denotes a processor, and disks are shown as cylinders):

- **Shared memory**. All the processors share a common memory (Figure 20.5a).

- **Shared disk**. A set of nodes that share a common set of disks; each node has its own processor and memory (Figure 20.5b). Shared-disk systems are sometimes called **clusters**.
- **Shared nothing**. A set of nodes that share neither a common memory nor common disk (Figure 20.5c).
- **Hierarchical**. This model is a hybrid of the preceding three architectures (Figure 20.5d). This model is the most widely used model today.

In Section 20.4.5 through Section 20.4.8, we elaborate on each of these models.

Note that the interconnection networks are shown in an abstract manner in Figure 20.5. Do not interpret the interconnection networks shown in the figures as necessarily being a bus; in fact other interconnection networks are used in practice. For example, mesh networks are used within a processor, and tree-like networks are often used to interconnect nodes.

20.4.5 Shared Memory

In a **shared-memory** architecture, the processors have access to a common memory, typically through an interconnection network. Disks are also shared by the processors. The benefit of shared memory is extremely efficient communication between processes—data in shared memory can be accessed by any process without being moved with software. A process can send messages to other processes much faster by using memory writes (which usually take less than a microsecond) than by sending a message through a communication mechanism.

Multicore processors with 4 to 8 cores are now common not just in desktop computers, but even in mobile phones. High-end processing systems such as Intel's Xeon processor have up to 28 cores per CPU, with up to 8 CPUs on a board, while the Xeon Phi coprocessor systems contain around 72 cores, as of 2018, and these numbers have been increasing steadily. The reason for the increasing number of cores is that the sizes of features such as logic gates in integrated circuits has been decreasing steadily, allowing more gates to be packed in a single chip. The number of transistors that can be accommodated on a given area of silicon has been doubling approximately every 1 1/2 to 2 years.[4]

Since the number of gates required for a processor core has not increased correspondingly, it makes sense to have multiple processors on a single chip. To maintain a distinction between on-chip multiprocessors and traditional processors, the term **core** is used for an on-chip processor. Thus, we say that a machine has a multicore processor.

[4]Gordon Moore, cofounder of Intel, predicted such an exponential growth in the number of transistors back in the 1960s; his prediction is popularly known as **Moore's law**, even though, technically, it is not a *law*, but rather an observation and a prediction. In earlier decades, processor speeds also increased along with the decrease in the feature sizes, but that trend ended in the mid-2000s since processor clock frequencies beyond a few gigahertz could not be attained without unreasonable increase in power consumption and heat generation. Moores's law is sometimes erroneously interpreted to have predicted exponential increases in processor speeds.

All the cores on a single processor typically access a shared memory. Further, a system can have multiple processors which can share memory. Another effect of the increasing number of gates has been the steady increase in the size of main memory as well as a decrease in cost, per-byte, of main memory.

Given the availability of multicore processors at a low cost, as well as the concurrent availability of very large amounts of memory at a low cost, shared-memory parallel processing has become increasingly important in recent years.

20.4.5.1 Shared-Memory Architectures

In earlier generation architectures, processors were connected to memory via a bus, with all processor cores and memory banks sharing a single bus. A downside of shared-memory accessed via a common bus is that the bus or the interconnection network becomes a bottleneck, since it is shared by all processors. Adding more processors does not help after a point, since the processors will spend most of their time waiting for their turn on the bus to access memory.

As a result, modern shared-memory architectures associate memory directly with processors; each processor has locally connected memory, which can be accessed very quickly; however, each processor can also access memory associated with other processors; a fast interprocessor communication network ensures that data are fetched with relatively low overhead. Since there is a difference in memory access speed depending on which part of memory is accessed, such an architecture is often referred to as **non-uniform memory architecture** (**NUMA**).

Figure 20.6 shows a conceptual architecture of a modern shared-memory system with multiple processors; note that each processor has a bank of memory directly connected to it, and the processors are linked by a fast interconnect system; processors are also connected to I/O controllers which interface with external storage.

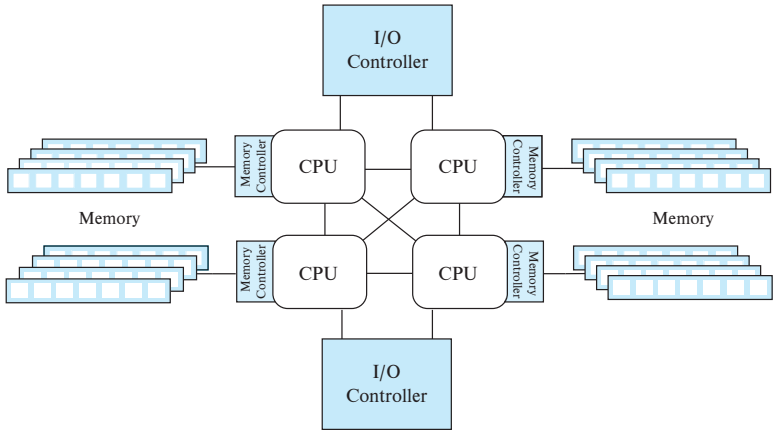

Figure 20.6 Architecture of a modern shared-memory system.

Because shared-memory architectures require specialized high-speed interconnects between cores and between processors, the number of cores/processors that can be interconnected in a shared-memory system is relatively small. As a result, the scalability of shared-memory parallelism is limited to at most a few hundred cores.

Processor architectures include cache memory, since access to cache memory is much faster than access to main memory (cache can be accessed in a few nanoseconds compared to nearly a hundred nanoseconds for main memory). Large cache memory is particularly important in shared-memory architectures, since a large cache can help minimize the number of accesses to shared memory.

If an instruction needs to access a data item that is not in cache, it must be fetched from main memory. Because main memory is much slower than processors, a significant amount of potential processing speed may be lost while a core waits for data from main memory. These waits are referred to as **cache misses**.

Many processor architectures support a feature called **hyper-threading**, or **hardware threads**, where a single physical core appears as two or more logical cores or threads. Different processes could be mapped to different logical cores. Only one of the logical cores corresponding to a single physical core can actually execute at any time. But the motivation for logical cores is that if the code running on one logical core blocks on a cache miss, waiting for data to be fetched from memory, the hardware of the physical core can start execution of one of the other logical cores instead of idling while waiting for data to be fetched from memory.

A typical multicore processor has multiple levels of cache, with the L1 cache being fastest to access, but also the smallest; lower cache levels such as L2 and L3 are slower (although still much faster than main memory) but considerably larger than the L1 cache. Lower cache levels are usually shared between multiple cores on a single processor. In the cache architecture shown in Figure 20.7, the L1 and L2 caches are local to each of the 4 cores, while the L3 cache is shared by all cores of the processor. data are read into, or written from, cache in units of a **cache line**, which typically consists of 64 consecutive bytes.

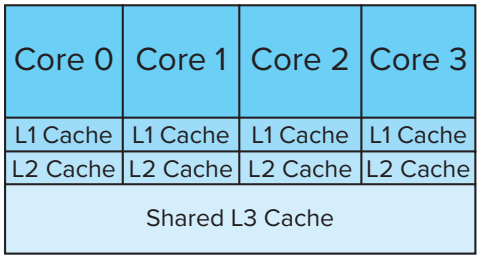

Figure 20.7 Multilevel cache system.

20.4.5.2 Cache Coherency

Cache coherency is an issue whenever there are multiple cores or processors, each with its own cache. An update done on one core may not be seen by another core, if the local cache on the second core contains an old value of the affected memory location. Thus, whenever an update occurs to a memory location, copies of the content of that memory location that are cached on other caches must be invalidated.

Such invalidation is done lazily in many processor architectures; that is, there may be some time lag between a write to a cache and the dispatch of invalidation messages to other caches; in addition there may be a further lag in processing invalidation messages that are received at a cache. (Requiring immediate invalidation to be done always can cause a significant performance penalty, and thus it is not done in current-generation systems.) Thus, it is quite possible for a write to happen on one processor, and a subsequent read on another processor may not see the updated value.

Such a lack of cache coherency can cause problems if a process expects to see an updated memory location but does not. Modern processors therefore support **memory barrier** instructions, which ensure certain orderings between load/store operations before the barrier and those after the barrier. For example, the **store barrier** instruction (sfence) on the x86 architecture forces the processor to wait until invalidation messages are sent to all caches for all updates done prior to the instruction, before any further load/store operations are issued. Similarly, the **load barrier** instruction (lfence) ensures all received invalidation messages have been applied before any further load/store operations are issued. The mfence instruction does both of these tasks.

Memory barrier instructions must be used with interprocess synchronization protocols to ensure that the protocols execute correctly. Without the use of memory barriers, if the caches are not "strongly" coherent, the following scenario can happen. Consider a situation where a process $P1$ updates memory location A first, then location B; a concurrent process $P2$ running on a different core or processor reads B first and then reads A. With a coherent cache, if $P2$ sees the updated value of B, it must also see the updated value of A. However, in the absence of cache coherence, the writes may be propagated out of order, and $P2$ may thus see the updated value of B but the old value of A. While many architectures disallow out-of-order propagation of writes, there are other subtle errors that can occur due to lack of cache coherency. However, executing sfence instructions after each of these writes and lfence before each of the reads will always ensure that reads see a cache coherent state. As a result, in the above example, the updated value of B will be seen only if the updated value of A is seen.

It is worth noting that programs do not need to include any extra code to deal with cache coherency, as long as they acquire locks before accessing data, and release locks only after performing updates, since lock acquire and release functions typically include the required memory barrier instructions. Specifically, an sfence instruction is executed as part of the lock release code, before the data item is actually unlocked. Similarly an lfence is executed right after locking a data item, as part of the lock acquisition

function, and is thus executed before the item is read. Thus, the reader is guaranteed to see the most recent value written to the data item.

Synchronization primitives supported in a variety of languages also internally execute memory barrier instructions; as a result, programmers who use these primitives need not be concerned about lack of cache coherency.

It is also interesting to note that many processor architectures use a form of hardware-level shared and exclusive locking of memory locations to ensure cache coherency. A widely used protocol, called the MESI protocol, can be understood as follows: Locking is done at the level of cache lines, containing multiple memory locations, instead of supporting locks on individual memory locations, since cache lines are the units of cache access. Locking is implemented in the hardware, rather than in software, to provide the required high performance.

The MESI protocol keeps track of the state of each cache line, which can be Modified (updated after exclusive locking), Exclusive locked (locked but not yet modified, or already written back to memory), Share locked, or Invalid. A read of a memory location automatically acquires a shared lock on the cache line containing that location, while a memory write gets an exclusive lock on the cache line before performing the write. In contrast to database locks, memory lock requests do not wait; instead they immediately revoke conflicting locks. Thus, an exclusive lock request automatically invalidates all cached copies of the cache line and revokes all shared locks on the cache line. Symmetrically, a shared lock request causes any existing exclusive lock to be revoked and then fetches the latest copy of the memory location into cache.

In principle, it is possible to ensure "strong" cache coherency with such a locking-based cache coherence protocol, making memory barrier instructions redundant. However, many implementations include some optimizations that speed up processing, such as allowing delayed delivery of invalidation messages, at the cost of not guaranteeing cache coherence. As a result, memory barrier instructions are required on many processor architectures to ensure cache coherency.

20.4.6 Shared Disk

In the **shared-disk** model, each node has its own processors and memory, but all nodes can access all disks directly via an interconnection network. There are two advantages of this architecture over a shared-memory architecture. First, a shared-disk system can scale to a larger number of processors than a shared-memory system. Second, it offers a cheap way to provide a degree of **fault tolerance**: If a node fails, the other nodes can take over its tasks, since the database is resident on disks that are accessible from all nodes.

We can make the disk subsystem itself fault tolerant by using a RAID architecture, as described in Chapter 12, allowing the system to function even if individual disks fail. The presence of a large number of storage devices in a RAID system also provides some degree of I/O parallelism.

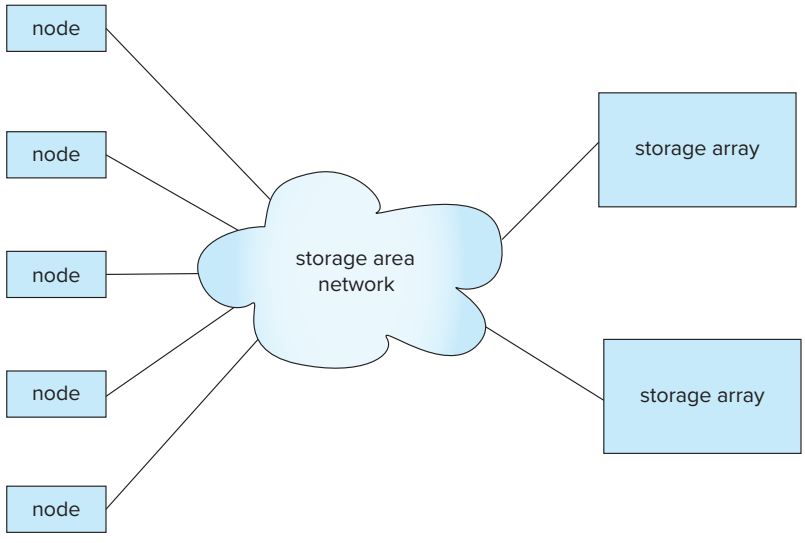

Figure 20.8 Storage-area network.

A **storage-area network** (SAN) is a high-speed local-area network designed to connect large banks of storage devices (disks) to nodes that use the data (see Figure 20.8). The storage devices physically consist of an array of multiple disks but provide a view of a logical disk, or set of disks, that hides the details of the underlying disks. For example, a logical disk may be much larger than any of the physical disks, and a logical disk's size can be increased by adding more physical disks. The processing nodes can access disks as if they are local disks, even though they are physically separate.

Storage-area networks are usually built with redundancy, such as multiple paths between nodes, so if a component such as a link or a connection to the network fails, the network continues to function.

Storage-area networks are well suited for building shared-disk systems. The shared-disk architecture with storage-area networks has found acceptance in applications that do not need a very high degree of parallelism but do require high availability.

Compared to shared-memory systems, shared-disk systems can scale to a larger number of processors, but communication across nodes is slower (up to a few milliseconds in the absence of special-purpose hardware for communication), since it has to go through a communication network.

One limitation of shared-disk systems is that the bandwidth of the network connection to storage in a shared-disk system is usually less than the bandwidth available to access local storage. Thus, storage access can become a bottleneck, limiting scalability.

20.4.7 Shared Nothing

In a **shared-nothing** system, each node consists of a processor, memory, and one or more disks. The nodes communicate by a high-speed interconnection network. A node

functions as the server for the data on the disk or disks that the node owns. Since local disk references are serviced by local disks at each node, the shared-nothing model overcomes the disadvantage of requiring all I/O to go through a single interconnection network.

Moreover, the interconnection networks for shared-nothing systems, such as the tree-like interconnection network, are usually designed to be scalable, so their transmission capacity increases as more nodes are added. Consequently, shared-nothing architectures are more scalable and can easily support a very large number of nodes.

The main drawbacks of shared-nothing systems are the costs of communication and of nonlocal disk access, which are higher than in a shared-memory or shared-disk architecture since sending data involves software interaction at both ends.

Due to their high scalability, shared-nothing architectures are widely used to deal with very large data volumes, supporting scalability to thousands of nodes, or in extreme cases, even to tens of thousands of nodes.

20.4.8 Hierarchical

The **hierarchical architecture** combines the characteristics of shared-memory, shared-disk, and shared-nothing architectures. At the top level, the system consists of nodes that are connected by an interconnection network and do not share disks or memory with one another. Thus, the top level is a shared-nothing architecture. Each node of the system could actually be a shared-memory system with a few processors. Alternatively, each node could be a shared-disk system, and each of the systems sharing a set of disks could be a shared-memory system. Thus, a system could be built as a hierarchy, with shared-memory architecture with a few processors at the base, and a shared-nothing architecture at the top, with possibly a shared-disk architecture in the middle. Figure 20.5d illustrates a hierarchical architecture with shared-memory nodes connected together in a shared-nothing architecture.

Parallel database systems today typically run on a hierarchical architecture, where each node supports shared-memory parallelism, with multiple nodes interconnected in a shared-nothing manner.

20.5 Distributed Systems

In a **distributed database system**, the database is stored on nodes located at geographically separated **sites**. The nodes in a distributed system communicate with one another through various communication media, such as high-speed private networks or the internet. They do not share main memory or disks. The general structure of a distributed system appears in Figure 20.9.

The main differences between shared-nothing parallel databases and distributed databases include the following:

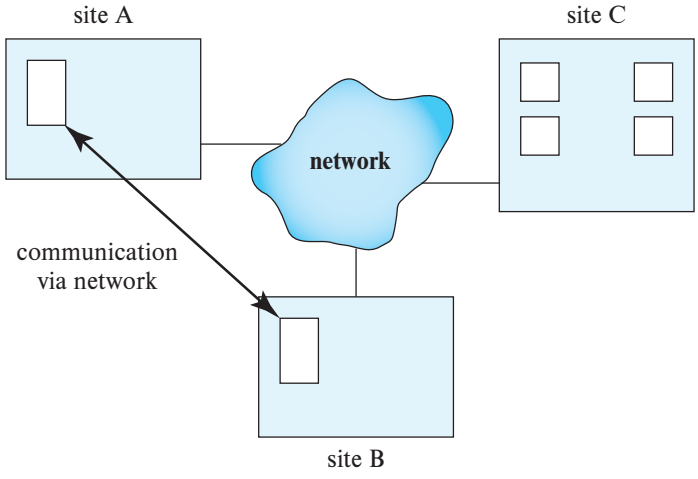

Figure 20.9 A distributed system.

- Distributed databases have sites that are geographically separated. As a result, the network connections have lower bandwidth, higher latency, and greater probability of failures, as compared to networks within a single data center.
  Systems built on distributed databases therefore need to be aware of network latency, and failures, as well as of physical data location. We discuss these issues later in this section. In particular, it is often desirable to keep a copy of the data at a data center close to the end user.
- Parallel database systems address the problem of node failure. However, some failures, particularly those due to earthquakes, fires, or other natural disasters, may affect an entire data center, causing failure of a large number of nodes. Distributed database systems need to continue working even in the event of failure of an entire data center, to ensure high availability. This requires replication of data across geographically separated data centers, to ensure that a common natural disaster does not affect all the data centers. Replication and other techniques to ensure high availability are similar in both parallel and distributed databases, although implementation details may differ.
- Distributed databases may be separately administered, with each site retaining some degree of autonomy of operation. Such databases are often the result of the integration of existing databases to allow queries and transactions to cross database boundaries. However, distributed databases that are built for providing geographic distribution, versus those built by integrating existing databases, may be centrally administered.
- Nodes in a distributed database tend to vary more in size and function, whereas parallel databases tend to have nodes that are of similar capacity.

- In a distributed database system, we differentiate between local and global transactions. A **local transaction** is one that accesses data only from nodes where the transaction was initiated. A **global transaction**, on the other hand, is one that either accesses data in a node different from the one at which the transaction was initiated, or accesses data in several different nodes.

Web-scale applications today run on data management systems that combine support for parallelism and distribution. Parallelism is used within a data center to handle high loads, while distribution across data centers is used to ensure high availability even in the event of natural disasters. At the lower end of functionality, such systems may be distributed data storage systems that support only limited functionality such as storage and retrieval of data by key, and they may not support schemas, query languages, or transactions; all such higher-level functionality has to be managed by the applications. At the higher end of functionality, there are distributed database systems that support schemas, query language, and transactions. However, one characteristic of such systems is that they are centrally administered.

In contrast, distributed databases that are built by integrating existing database systems have somewhat different characteristics.

- **Sharing data.** The major advantage in building a distributed database system is the provision of an environment where users at one site may be able to access the data residing at other sites. For instance, in a distributed university system, where each campus stores data related to that campus, it is possible for a user in one campus to access data in another campus. Without this capability, the transfer of student records from one campus to another campus would have to rely on some external mechanism.
- **Autonomy.** The primary advantage of sharing data by means of data distribution is that each site can retain a degree of control over data that are stored locally. In a centralized system, the database administrator of the central site controls the database. In a distributed system, there is a global database administrator responsible for the entire system. A part of these responsibilities is delegated to the local database administrator for each site. Depending on the design of the distributed database system, each administrator may have a different degree of **local autonomy**.

In a **homogeneous distributed database** system, nodes share a common global schema (although some relations may be stored only at some nodes), all nodes run the same distributed database-management software, and the nodes actively cooperate in processing transactions and queries.

However, in many cases a distributed database has to be constructed by linking together multiple already-existing database systems, each with its own schema and possibly running different database-management software. The sites may not be aware of one another, and they may provide only limited facilities for cooperation in query and transaction processing. Such systems are sometimes called **federated database systems** or **heterogeneous distributed database systems**.

CHAPTER 21

Parallel and Distributed Storage

As we discussed in Chapter 20, parallelism is used to provide speedup, where queries are executed faster because more resources, such as processors and disks, are provided. Parallelism is also used to provide scaleup, where increasing workloads are handled without increased response time, via an increase in the degree of parallelism.

In this chapter, we discuss techniques for data storage and indexing in parallel database systems.

21.1 Overview

We first describe, in Section 21.2 and Section 21.3, how to partition data amongst multiple nodes. We then discuss, in Section 21.4, replication of data and parallel indexing (in Section 21.5). Our description focuses on shared-nothing parallel database systems, but the techniques we describe are also applicable to distributed database systems, where data are stored in a geographically distributed manner.

File systems that run on a large number of nodes, called distributed file systems, are a widely used way to store data in a parallel system. We discuss distributed file systems in Section 21.6.

In recent years parallel data storage and indexing techniques have been extensively used for storage of nonrelational data, including unstructured text data, and semi-structured data in XML, JSON, or other formats. Such data are often stored in parallel **key-value stores**, which store data items with an associated key. The techniques we describe for parallel storage of relational data can also be used for key-value stores which are discussed in Section 21.7. We use the term **data storage system** to refer to both key-value stores, and the data storage and access layer of parallel database systems.

Query processing in parallel and distributed databases is discussed in Chapter 22, while and transaction processing in parallel and distributed databases is discussed in Chapter 23.

21.2 Data Partitioning

In its simplest form, **I/O parallelism** refers to reducing the time required to retrieve data from disk by partitioning the data over multiple disks.[1]

At the lowest level, RAID systems allow blocks to be partitioned across multiple disks, allowing them to be accessed in parallel. Blocks are usually allocated to different disks in a round-robin fashion, as we saw in Section 12.5. For example, if there are n disks numbered 0 to $n - 1$, round-robin allocation assigns block i to disk i mod n. However, the block-level partitioning techniques supported by RAID systems do not offer any control in terms of which tuples of a relation are stored on which disk or node. Therefore, parallel database systems typically do not use block-level partitioning and instead perform partitioning at the level of tuples.

In systems with multiple nodes (computers), each with multiple disks, partitioning can potentially be specified to the level of individual disks. However, parallel database systems typically focus on partitioning data across nodes and leave it to the operating system on each node to decide on assigning blocks to disks within the node.

In a parallel storage system, the tuples of a relation are partitioned (divided) among many nodes, so that each tuple resides on one node; such partitioning is referred to as **horizontal partitioning**. Several partitioning strategies have been proposed for horizontal partitioning, which we study next.

We note that *vertical partitioning*, discussed in Section 13.6 in the context of columnar storage, is orthogonal to horizontal partitioning. (As an example of vertical partitioning, a relation $r(A, B, C, D)$ where A is a primary key, may be vertically partitioned into $r(A, B)$ and $r(A, C, D)$, if many queries require B values, while C and D values are large in size and not required for many queries.) Once tuples are horizontally partitioned, they may be stored in a vertically partitioned manner at each node.

We also note that several database vendors use the term *partitioning* to denote the partitioning of tuples of a relation r into multiple physical relations $r_1, r_2, \ldots, r_n$, where all the physical relations r_i are stored in a single node. The relation r is not stored, but treated as a view defined by the query $r_1 \cup r_2 \cup \ldots \cup r_n$. Such *intra-node partitioning* of a relation is typically used to ensure that frequently accessed tuples are stored separately from infrequently accessed tuples and is different from horizontal partitioning across nodes. Intra-node partitioning is described in more detail in Section 25.1.4.3.

In the rest of this chapter, as well as in subsequent chapters, we use the term *partitioning* to refer to *horizontal partitioning* across multiple nodes.

21.2.1 Partitioning Strategies

We present three basic *data-partitioning strategies* for partitioning tuples. Assume that there are n nodes, $N_1, N_1, \ldots, N_n$, across which the data are to be partitioned.

[1] As in earlier chapters, we use the term *disk* to refer to persistent storage devices, such as magnetic hard disks and solid-state drives.

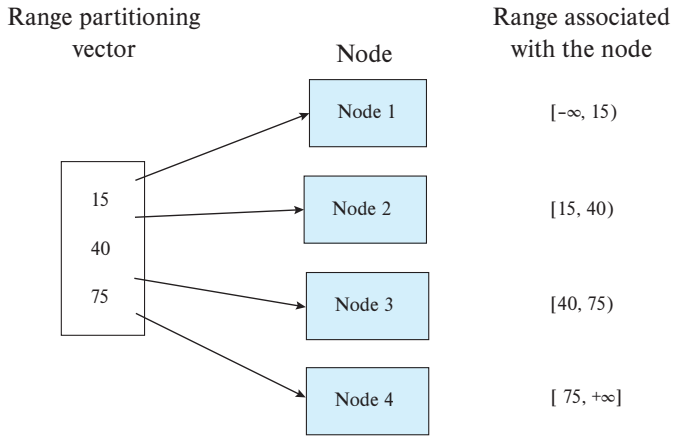

Figure 21.1 Example of range partitioning vector.

- **Round-robin**. This strategy scans the relation in any order and sends the ith tuple fetched during the scan to node number $N_{((i-1) \bmod n)+1}$. The round-robin scheme ensures an even distribution of tuples across nodes; that is, each node has approximately the same number of tuples as the others.
- **Hash partitioning**. This declustering strategy designates one or more attributes from the given relation's schema as the partitioning attributes. A hash function is chosen whose range is $\{1, 2, \ldots, n\}$. Each tuple of the original relation is hashed on the partitioning attributes. If the hash function returns i, then the tuple is placed on node N_i.[2]
- **Range partitioning**. This strategy distributes tuples by assigning contiguous attribute-value ranges to each node. It chooses a partitioning attribute, A, and a **partitioning vector** $[v_1, v_2, \ldots, v_{n-1}]$, such that, if $i < j$, then $v_i < v_j$. The relation is partitioned as follows: Consider a tuple t such that $t[A] = x$. If $x < v_1$, then t goes on node N_1. If $x \geq v_{n-1}$, then t goes on node N_n. If $v_i \leq x < v_{i+1}$, then t goes on node N_{i+1}.

Figure 21.1 shows an example of a range partitioning vector. In the example in the figure, values less than 15 are mapped to Node 1. Values in the range $[15, 40)$, i.e., values ≥ 15 but < 40. are mapped to Node 2; Values in the range $[40, 75)$, i.e., values ≥ 40 but < 75, are mapped to Node 3, while values > 75 are mapped to Node 4.

We now consider how partitioning is maintained when a relation is updated.

1. When a tuple is inserted into a relation, it is sent to the appropriate node based on the partitioning strategy.

[2]Hash-function design is discussed in Section 24.5.1.1.

2. If a tuple is deleted, its location is first found based on the value of its partitioning attribute (for round-robin, all partitions are searched). The tuple is then deleted from wherever it is located.
3. If a tuple is updated, its location is not affected if either round-robin partitioning is used or if the update does not affect a partitioning attribute.
   However, if range partitioning or hash partitioning is used, and the update affects a partitioning attribute, the location of the tuple may be affected. In this case:
   a. The original tuple is deleted from the original location, and
   b. The updated tuple is inserted and sent to the appropriate node based on the partitioning strategy used.

21.2.2 Comparison of Partitioning Techniques

Once a relation has been partitioned among several nodes, we can retrieve it in parallel, using all the nodes. Similarly, when a relation is being partitioned, it can be written to multiple nodes in parallel. Thus, the transfer rates for reading or writing an entire relation are much faster with I/O parallelism than without it. However, reading an entire relation, or *scanning a relation*, is only one kind of access to data. Access to data can be classified as follows:

1. Scanning the entire relation.
2. Locating a tuple associatively (e.g., *employee_name* = "Campbell"); these queries, called **point queries**, seek tuples that have a specified value for a specific attribute.
3. Locating all tuples for which the value of a given attribute lies within a specified range (e.g., 10000 < *salary* < 20000); these queries are called **range queries**.

The different partitioning techniques support these types of access at different levels of efficiency:

- **Round-robin.** The scheme is ideally suited for applications that wish to read the entire relation sequentially for each query. With this scheme, both point queries and range queries are complicated to process, since each of the n nodes must be used for the search.
- **Hash partitioning.** This scheme is best suited for point queries based on the partitioning attribute. For example, if a relation is partitioned on the *telephone_number* attribute, then we can answer the query "Find the record of the employee with *telephone_number* = 555-3333" by applying the partitioning hash function to 555-3333 and then searching that node. Directing a query to a single node saves the start-up cost of initiating a query on multiple nodes and leaves the other nodes free to process other queries.

Hash partitioning is also useful for sequential scans of the entire relation. If the hash function is a good randomizing function, and the partitioning attributes form a key of the relation, then the number of tuples in each of the nodes is approximately the same, without much variance. Hence, the time taken to scan the relation is approximately $1/n$ of the time required to scan the relation in a single node system.

The scheme, however, is not well suited for point queries on nonpartitioning attributes. Hash-based partitioning is also not well suited for answering range queries, since, typically, hash functions do not preserve proximity within a range. Therefore, all the nodes need to be scanned for range queries to be answered.

- **Range partitioning.** This scheme is well suited for point and range queries on the partitioning attribute. For point queries, we can consult the partitioning vector to locate the node where the tuple resides. For range queries, we consult the partitioning vector to find the range of nodes on which the tuples may reside. In both cases, the search narrows to exactly those nodes that might have any tuples of interest.

  An advantage of this feature is that, if there are only a few tuples in the queried range, then the query is typically sent to one node, as opposed to all the nodes. Since other nodes can be used to answer other queries, range partitioning results in higher throughput while maintaining good response time. On the other hand, if there are many tuples in the queried range (as there are when the queried range is a larger fraction of the domain of the relation), many tuples have to be retrieved from a few nodes, resulting in an I/O bottleneck (hot spot) at those nodes. In this example of **execution skew**, all processing occurs in one—or only a few—partitions. In contrast, hash partitioning and round-robin partitioning would engage all the nodes for such queries, giving a faster response time for approximately the same throughput.

The type of partitioning also affects other relational operations, such as joins, as we shall see in Section 22.3 and Section 22.4.1.

Thus, the choice of partitioning technique also depends on the operations that need to be executed. In general, hash partitioning or range partitioning are preferred to round-robin partitioning.

Partitioning is important for large relations. Large databases that benefit from parallel storage often have some small relations. Partitioning is not a good idea for such small relations, since each node would end up with just a few tuples. Partitioning is worthwhile only if each node would contain at least a few disk blocks worth of data. Small relations are best left unpartitioned, while medium-sized relations could be partitioned across some of the nodes, rather than across all the nodes, in a large system.

21.3 Dealing with Skew in Partitioning

When a relation is partitioned (by a technique other than round-robin), there may be a skew in the distribution of tuples, with a high percentage of tuples placed in some

partitions and fewer tuples in other partitions. Such an imbalance in the distribution of data is called **data distribution skew**. Data distribution skew may be caused by one of two factors.

- **Attribute-value skew**, which refers to the fact that some values appear in the partitioning attributes of many tuples. All the tuples with the same value for the partitioning attribute end up in the same partition, resulting in skew.
- **Partition skew**, which refers to the fact that there may be load imbalance in the partitioning, even when there is no attribute skew.

Attribute-value skew can result in skewed partitioning regardless of whether range partitioning or hash partitioning is used. If the partition vector is not chosen carefully, range partitioning may result in partition skew. Partition skew is less likely with hash partitioning if a good hash function is chosen.

As Section 20.4.2 noted, even a small skew can result in a significant decrease in performance. Skew becomes an increasing problem with a higher degree of parallelism. For example, if a relation of 1000 tuples is divided into 10 parts, and the division is skewed, then there may be some partitions of size less than 100 and some partitions of size more than 100; if even one partition happens to be of size 200, the speedup that we would obtain by accessing the partitions in parallel is only 5, instead of the 10 for which we would have hoped. If the same relation has to be partitioned into 100 parts, a partition will have 10 tuples on an average. If even one partition has 40 tuples (which is possible, given the large number of partitions) the speedup that we would obtain by accessing them in parallel would be 25, rather than 100. Thus, we see that the loss of speedup due to skew increases with parallelism.

In addition to skew in the distribution of tuples, there may be **execution skew** even if there is no skew in the distribution of tuples, if queries tend to access some partitions more often than others. For example, suppose a relation is partitioned by the timestamp of the tuples, and most queries refer to recent tuples; then, even if all partitions contain the same number of tuples, the partition containing recent tuples would experience a significantly higher load.

In the rest of this section, we consider several approaches to handling skew.

21.3.1 Balanced Range-Partitioning Vectors

Data distribution skew in range partitioning can be avoided by choosing a **balanced range-partitioning vector**, which evenly distributes tuples across all nodes.

A balanced range-partitioning vector can be constructed by sorting, as follows: The relation is first sorted on the partitioning attributes. The relation is then scanned in sorted order. After every $1/n$ of the relation has been read, the value of the partitioning attribute of the next tuple is added to the partition vector. Here, n denotes the number of partitions to be constructed.

The main disadvantage of this method is the extra I/O overhead incurred in doing the initial sort. The I/O overhead for constructing balanced range-partitioning vectors can be reduced by using a precomputed frequency table, or **histogram**, of the attribute values for each attribute of each relation. Figure 21.2 shows an example of a histogram for an integer-valued attribute that takes values in the range 1 to 25. It is straightforward to construct a balanced range-partitioning function given a histogram on the partitioning attributes. A histogram takes up only a little space, so histograms on several different attributes can be stored in the system catalog. If the histogram is not stored, it can be computed approximately by sampling the relation, using only tuples from a randomly chosen subset of the disk blocks of the relation. Using a random sample allows the histogram to be constructed in much less time than it would take to sort the relation.

The preceding approach for creating range-partitioning vectors addresses data-distribution skew; extensions to handle execution skew are left as an exercise for the reader (Exercise 21.3).

A drawback of the above approach is that it is *static*: the partitioning is decided at some point and is not automatically updated as tuples are inserted, deleted, or updated. The partitioning vectors can be recomputed, and the data repartitioned, whenever the system detects skew in data distribution. However, the cost of repartitioning can be quite large, and doing it periodically would introduce a high load which can affect normal processing. Dynamic techniques for avoiding skew, which can adapt in a continuous and less disruptive fashion, are discussed in Section 21.3.2 and in Section 21.3.3.

21.3.2 Virtual Node Partitioning

Another approach to minimizing the effect of skew is to use *virtual nodes*. In the **virtual nodes** approach, we pretend there are several times as many *virtual nodes* as the number

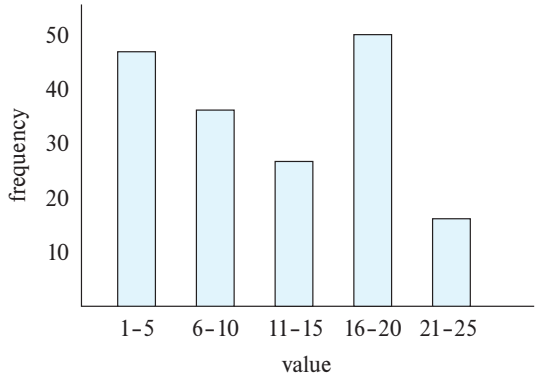

Figure 21.2 Example of histogram.

of real nodes. Any of the partitioning techniques described earlier can be used, but to map tuples and work to virtual nodes instead of to real nodes.[3]

Virtual nodes, in turn, are mapped to real nodes. One way to map virtual nodes to real nodes is round-robin allocation; thus, if there are n real nodes numbered 1 to n, virtual node i is mapped to real node $((i-1) mod n) + 1$. The idea is that even if one range had many more tuples than the others because of skew, these tuples would get split across multiple virtual nodes ranges. Round-robin allocation of virtual nodes to real nodes would distribute the extra work among multiple real nodes, so that one node does not have to bear all the burden.

A more sophisticated way of doing the mapping is by tracking the number of tuples in each virtual node, and the load (e.g., the number of accesses per second) on each virtual node. Virtual nodes are then mapped to real nodes in a way that balances the number of stored tuples as well as the load across the real nodes. Thus, data-distribution skew and execution skew can be minimized.

The system must then record this mapping and use it to route accesses to the correct real node. If virtual nodes are numbered by consecutive integers, this mapping can be stored as an array *virtual_to_real_map[]*, with m entries, where there are m virtual nodes; the ith element of this array stores the real node to which virtual node i is mapped.

Yet another benefit of the virtual node approach is that it allows **elasticity of storage**, that is, as the load on the system increases it is possible to add more resources (nodes) to the system to handle the load. When a new node is added, some of the virtual nodes are *migrated* to the new real node, which can be done without affecting any of the other virtual nodes. If the amount of data mapped to each virtual node is small, the migration of a virtual node from one node to another can be done relatively fast, minimizing disruption.

21.3.3 Dynamic Repartitioning

While the virtual-node approach can reduce skew with range partitioning as well as hash partitioning, it does not work very well if the data distribution changes over time, resulting in some virtual nodes having a very large number of tuples, or a very high execution load. For example, if partitioning was done by timestamps of records, the last timestamp range would get an increasing number of records, as more records are inserted, while other ranges would not get any new records. Thus, even if the initial partitioning is balanced, it could become increasingly skewed over time.

Skew can be dealt with by recomputing the partitioning scheme entirely. However, repartitioning the data based on the new partitioning scheme would, in general, be a very expensive operation. In the preceding example, we would end up moving a significant number of records from each partition to a partition that precedes it in timestamp

[3]The virtual node approach is also called the **virtual processor** approach, a term used in earlier editions of this book; since the *term virtual* processor is now commonly used in a different sense in the context of virtual machines, we now use the term *virtual node*.

order. When dealing with large amounts of data, such repartitioning would be unreasonably expensive.

Dynamic repartitioning can be done in an efficient manner by instead exploiting the virtual node scheme. The basic idea is to split a virtual node into two virtual nodes when it has too many tuples, or too much load; the idea is very similar to a B^+-tree node being split into two nodes when it is overfull. One of the newly created virtual nodes can then be migrated to a different node to rebalance the data stored at each node, or the load at each node.

Considering the preceding example, if the virtual node corresponding to a range of timestamps 2017-01-01 to *MaxDate* were to become overfull, the partition could be split into two partitions. For example, if half the tuples in this range have timestamps less than 2018-01-01, one partition would have timestamps from 2017-01-01 to less than 2018-01-01, and the other would have tuples with timestamps from 2018-01-01 to *MaxDate*. To rebalance the number of tuples in a real node, we would just need to move one of the virtual nodes to a new real node.

Dynamic repartitioning in this way is very widely used in parallel databases and parallel data storage systems today. In data storage systems, the term **table** refers to a collection of data items. Tables are partitioned into multiple **tablets**. The number of tablets into which a table is divided is much larger than the number of real nodes in the system; thus tablets correspond to virtual nodes.

The system needs to maintain a **partition table**, which provides a mapping from the partitioning key ranges to a tablet identifier, as well as the real node on which the tablet data reside. Figure 21.3 shows an example of a partition table, where the partition key is a date. Tablet0 stores records with key value $<$ 2012-01-01. Tablet1 stores records with key values $\geq$ 2012-01-01, but $<$ 2013-01-01. Tablet2 stores records with key values $\geq$ 2013-01-01, but $<$ 2014-01-01, and so on. Finally, Tablet6 stores values $\geq$ 2017-01-01.

Read requests must specify a value for the partitioning attribute, which is used to identify the tablet which could contain a record with that key value; a request that does not specify a value for the partitioning attribute would have to be sent to all tablets. A read request is processed by using the partitioning key value v to identify the tablet

Value	Tablet ID	Node ID
2012-01-01	Tablet0	Node0
2013-01-01	Tablet1	Node1
2014-01-01	Tablet2	Node2
2015-01-01	Tablet3	Node2
2016-01-01	Tablet4	Node0
2017-01-01	Tablet5	Node1
MaxDate	Tablet6	Node1

Figure 21.3 Example of a partition table.

whose range of keys contains v, and then sending the request to the real node where the tablet resides. The request can be handled efficiently at that node by maintaining, for each tablet, an index on the partitioning key attribute.

Write, insert, and delete requests are processed similarly, by routing the requests to the correct tablet and real node, using the mechanism described above for reads.

The above scheme allows tablets to be split if they become too big; the key range corresponding to the tablet is split into two, with a newly created tablet getting half the key range. Records whose key range is mapped to the new tablet are then moved from the original tablet to the new tablet. The partition table is updated to reflect the split, so requests are then correctly directed to the appropriate tablet.

If a real node gets overloaded, either due to a large number of requests or due to too much data at the node, some of the tablets from the node can be moved to a different real node that has a lower load. Tablets can also be moved similarly in case one of the real nodes has a large amount of data, while another real node has less data. Finally, if a new real node joins a system, some tables can be moved from existing nodes to the new node. Whenever a tablet is moved to a different real node, the partition table is updated; subsequent requests will then be sent to the correct real node.

Figure 21.4 shows the partition table from Figure 21.3 after Tablet6, which had values $\geq$ 2017-01-01, has been split into two: Tablet6 now has values $\geq$ 2017-01-01, but $<$ 2018-01-01, while the new tablet, Tablet7, has values $\geq$ 2018-01-01. Such a split could be caused by a large number of inserts into Tablet6, making it very large; the split rebalances the sizes of the tablets.

Note also that Tablet1, which was in Node1, has now been moved to Node0 in Figure 21.4. Such a tablet move could be because Node1 is overloaded due to excessive data, or due to a high number of requests.

Most parallel data storage systems store the partition table at a **master** node. However, to support a large number of requests each second, the partition table is usually replicated, either to all client nodes that access data or to multiple **routers**. Routers accept read/write requests from clients and forward the requests to the appropriate

Value	Tablet ID	Node ID
2012-01-01	Tablet0	Node0
2013-01-01	Tablet1	Node0
2014-01-01	Tablet2	Node2
2015-01-01	Tablet3	Node2
2016-01-01	Tablet4	Node0
2017-01-01	Tablet5	Node1
2018-01-01	Tablet6	Node1
MaxDate	Tablet7	Node1

Figure 21.4 Example partition table after tablet split and tablet move.

real node containing the tablet/virtual nodes based on the key values specified in the request.

An alternative fully distributed approach is supported by a hash based partitioning scheme called **consistent hashing**. In the consistent hashing approach, keys are hashed to a large space, such as 32 bit integers. Further, node (or virtual node) identifiers are also hashed to the same space. A key k_i could be logically mapped to the node n_j whose hash value $h(n_j)$ is the highest value among all nodes satisfying $h(n_j) < h(k_i)$. But to ensure that every key is assigned to a node, hash values are treated as lying on a cycle similar to the face of a clock, where the maximum hash value *maxhash* is immediately followed by 0. Then, key k_i is then logically mapped to the node n_j whose hash value $h(n_j)$ is the closest among all nodes, when we move anti-clockwise in the circle from $h(k_i)$.

Distributed hash tables based on this idea have been developed where there is no need for either a master node or a router; instead each participating node keeps track of a few other peer nodes, and routing is implemented in a cooperative manner. New nodes can join the system, and integrate themselves by following specified protocols in a completely peer-to-peer manner, without the need for a master. See the Further Reading section at the end of the chapter for references providing further details.

21.4 Replication

With a large number of nodes, the probability that at least one node will malfunction in a parallel system is significantly greater than in a single-node system. A poorly designed parallel system will stop functioning if any node fails. Assuming that the probability of failure of a single node is small, the probability of failure of the system goes up linearly with the number of nodes. For example, if a single node would fail once every 5 years, a system with 100 nodes would have a failure every 18 days.

Parallel data storage systems must, therefore, be resilient to failure of nodes. Not only should data not be lost in the event of a node failure, but also, the system should continue to be *available*, that is, continue to function, even during such a failure.

To ensure tuples are not lost on node failure, tuples are replicated across at least two nodes, and often three nodes. If a node fails, the tuples that it stored can still be accessed from the other nodes where the tuples are replicated.[4]

Tracking the replicas at the level of individual tuples would result in significant overhead in terms of storage and query processing. Instead, replication is done at the level of partitions (tablets, nodes, or virtual nodes). That is, each partition is replicated; the locations of the partition replicas are recorded as part of the partition table.

Figure 21.5 shows a partition table with replication of tablets. Each tablet is replicated in two nodes.

[4]Caching also results in replication of data, but with the aim of speeding up access. Since data may be evicted from cache at any time, caching does not ensure availability in the event of failure.

Value	Tablet ID	Node ID
2012-01-01	Tablet0	Node0,Node1
2013-01-01	Tablet1	Node0,Node2
2014-01-01	Tablet2	Node2,Node0
2015-01-01	Tablet3	Node2,Node1
2016-01-01	Tablet4	Node0,Node1
2017-01-01	Tablet5	Node1,Node0
2018-01-01	Tablet6	Node1,Node2
MaxDate	Tablet7	Node1,Node2

Figure 21.5 Partition table of Figure 21.4 with replication.

The database system keeps track of failed nodes; requests for data stored at a failed node are automatically routed to the backup nodes that store a replica of the data. Issues of how to handle the case where one or more replicas are stored at a currently failed node are addressed briefly in Section 21.4.2, and in more detail later, in Section 23.4.

21.4.1 Location of Replicas

Replication to two nodes provides protection from data loss/unavailability in the event of single node failure, while replication to three nodes provides protection even in the event of two node failures. If all nodes where a partition is replicated fail, obviously there is no way to prevent data loss/unavailability. Systems that use low-cost commodity machines for data storage typically use three-way replication, while systems that use more reliable machines typically use two-way replication.

There are multiple possible failure modes in a parallel system. A single node could fail due to some internal fault. Further, it is possible for all the nodes in a rack to fail if there is some problem with the rack such as, for example, failure of power supply to the entire rack, or failure of the network switches in a rack, making all the nodes in the rack inaccessible. Further, there is a possibility of failure of an entire data center, for example, due to fire, flooding, or a large-scale power failure.

The location of the nodes where the replicas of a partition are stored must, therefore, be chosen carefully, to maximize the probability of at least one copy being accessible even during a failure. Such replication can be within a data center or across data centers.

- **Replication within a data center**: Since single node failures are the most common failure mode, partitions are often replicated to another node.
  With the tree-like interconnection topology commonly used in data center networks (described in Section 20.4.3) network bandwidth within a rack is much

higher than the network bandwidth between racks. As a result, replication to another node within the same rack as the first node reduces network demand on the network between racks. But to deal with the possibility of a rack failure, partitions are also replicated to a node on a different rack.

- **Replication across data centers**: To deal with the possibility of failure of an entire data center, partitions may also be replicated at one or more geographically separated data centers. Geographic separation is important to deal with disasters such as earthquakes or storms that may shut down all data centers in a geographic region.

  For many web applications, round-trip delays across a long-distance network can affect performance significantly, a problem that is increasing with the use of Ajax applications that require multiple rounds of communication between the browser and the application. To deal with this problem, users are connected with application servers that are closest to them geographically, and data replication is done in such a way that one of the replicas is in the same data center as (or at least, geographically close to) the application server.

Suppose all the partitions at a node N_1 are replicated at a single node N_2, and N_1 fails. Then, node N_2 will have to handle all the requests that would originally have gone to N_1, as well as requests routed to node N_2. As a result, node N_2 would have to perform twice as much work as other nodes in the system, resulting in execution skew during failure of node N_1.

To avoid this problem, the replicas of partitions residing at a node, say N_1, are spread across multiple other nodes. For example, consider a system with 10 nodes and two-way replication. Suppose node N_1 had one of the replicas of partitions p_1 through p_9. Then, the other replica of partitions p_1 could be stored on N_2, of p_2 on N_3, and so on to p_9 on N_{10}. Then in the event of failure of N_1, nodes N_2 through N_{10} would share the extra work equally, instead of burdening a single node with all the extra work.

21.4.2 Updates and Consistency of Replicas

Since each partition is replicated, updates made to tuples in a partition must be performed on all the replicas of the partition. For data that is never updated after it has been created, reads can be performed at any of the replicas, since all of them will have the same value. If a storage system ensures that all replicas are exclusive-locked and updated atomically (using, for example, the two-phase commit protocol which we will see in Section 23.2.1), reads of a tuple can be performed (after obtaining a shared lock) at any of the replicas and will see the most recent version of the tuple.

If data are updated, and replicas are not updated atomically, different replicas may temporarily have different values. Thus, a read may see a different value depending on which replica it accesses. Most applications require that read requests for a tuple must

receive the most recent version of the tuple; updates that are based on reading an older version could result in a *lost update problem*.

One way of ensuring that reads get the latest value is to treat one of the replicas of each partition as a **master replica**. All updates are sent to the master replica and are then propagated to other replicas. Reads are also sent to the master replica, so that reads get the latest version of any data item even if updates have not yet been applied to the other replicas.

If a master replica fails, a new master is assigned for that partition. It is important to ensure that every update operation performed by the old master has also been seen by the new master. Further, the old master may have updated some of the replicas, but not all, before it failed; the new master must complete the task of updating all the replicas. We discuss details in Section 23.6.2.

It is important to know which node is the (current) master for each partition. This information can be stored along with the partition table. Specifically, the partition table must record, in addition to the range of key values assigned to that partition, where the replicas of the partition are stored, and further which replica is currently the master.

Three solutions are commonly used to update replicas.

- The *two-phase commit* (2PC) protocol, which ensures that multiple updates performed by a transaction are applied atomically across multiple sites, is described in Section 23.2. This protocol can be used with replicas to ensure that an update is performed atomically on all replicas of a tuple.

  We assume for now that all replicas are available and can be updated. Issues of how to allow two-phase commit to continue execution in the presence of failures, when some replicas may not be reachable, are discussed in Section 23.4.

- Persistent messaging systems, described in Section 23.2.3, which guarantee that a message is delivered once it is sent. Persistent messaging systems can be used to update replicas as follows: An update to a tuple is registered as a persistent message, sent to all replicas of the tuple. Once the message is recorded, the persistent messaging system ensures it will be delivered to all replicas. Thus, all replicas will get the update, eventually; the property is known as **eventual consistency** of replicas.

  However, there may be a delay in message delivery, and during that time some replicas may have applied an update while others have not. To ensure that reads get a consistent version, reads are performed only at a master replica, where updates are made first. (If the site with a master replica of a tuple has failed, another replica can take over as the master replica after ensuring all pending persistent messages with updates have been applied.) Details are presented in Section 23.6.2.

- Protocols called *consensus protocols*, that allow updates of replicas to proceed even in the face of failures, when some replicas may not be reachable, can be used to manage update of replicas. Unlike the preceding protocols, consensus protocols can work even without a designated master replica. We study consensus protocols in Section 23.8.

21.5 Parallel Indexing

Indices in a parallel data storage system can be divided into two kinds: local and global indices. In the following discussion, when virtual node partitioning is used, the term *node* should be understood to mean virtual node (or equivalently, tablet).

- A **local index** is an index built on tuples stored in a particular node; typically, such an index would be built on all the partitions of a given relation. The index contents are stored on the same node as the data.
- A **global index** is an index built on data stored across multiple nodes; a global index can be used to efficiently find matching tuples, regardless of where the tuples are stored.

  While the contents of a global index could be stored at a single central location, such a scheme would result in poor scalability; as a result, the contents of a global index should partitioned across multiple nodes.

A **global primary index** on a relation is a global index on the attributes on which the tuples of the relation are partitioned. A global index on partitioning attribute K is constructed by merely creating local indices on K on each partition.

A query that is intended to retrieve tuples with a specific key value $k1$ for K can be answered by first finding which partition could hold the key value $k1$, and then using the local index in that partition to find the required tuples.

For example, suppose the *student* relation is partitioned on the attribute ID, and a global index is to be constructed on the attribute ID. All that is required is to construct a local index on ID on each partition. Figure 21.6(a) shows a global primary index on the *student* relation, on attribute ID; the local indices are not shown explicitly in the figure.

A query that is intended to retrieve tuples with a specific value for ID, say 557, can be answered by first using the partitioning function on ID to first find which partition could contain the specified ID value 557; the query is then sent to the corresponding node, which uses the local index on ID to locate the required tuples.

Note that ID is the primary key for the relation *student*; however, the above scheme would work even if the partitioning attribute were not the primary key. The scheme can be extended to handle range queries, provided the partitioning function is itself based on ranges of values; a partitioning scheme based on hashing cannot support range queries.

A **global secondary index** on a relation is a global index whose index attributes do not match the attributes on which the tuples are partitioned.

Suppose the partitioning attributes are K_p, while the index attributes are K_i, with $K_p \neq K_i$. One approach for answering a selection query on attributes K_i is as follows: If a local index is created on K_i on each partition of the relation, the query is sent to each

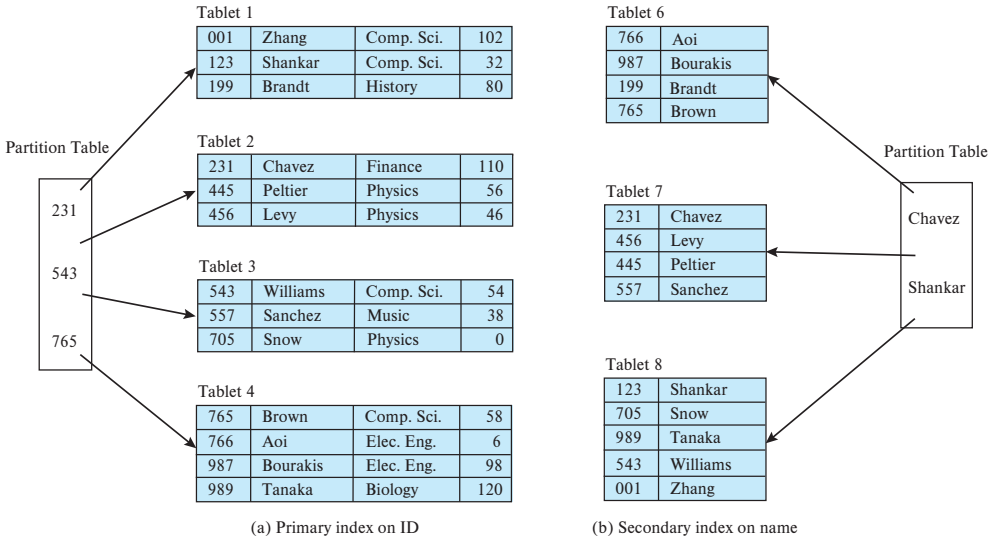

Figure 21.6 Global primary and secondary indices on *student* relation

partition, and the local index is used to find matching tuples. Such an approach using local indices is very inefficient if the number of partitions is large, since every partition has to be queried, even if only one or a few partitions contain matching tuples.

We now illustrate an efficient scheme for constructing a global secondary index by using an example. Consider again the *student* relation partitioned on the attribute ID, and suppose a global index is to be constructed on the attribute *name*. A simple way to construct such an index is to create a set of (*name*, ID) tuples, with one tuple per *student* tuple; let us call this set of tuples *index_name*. Now, the *index_name* tuples are partitioned on the attribute *name*. A local index on *name* is then constructed on each partition of *index_name*. In addition, a global index is created on the ID, which is the partitioning attribute. Figure 21.6(b) shows a secondary index on the *student* relation on attribute *name*; local indices are not explicitly shown in the figure.

Now, a query that needs to retrieve students with a given name can be handled by first examining the partition function of *index_name* to find which partition could store *index_name* tuples with the given name; the query is then sent to that partition, which uses the local index on *name* to find the corresponding ID value. Next, the global index on the ID value is used to find the required tuple.

Note that in the example above, the partitioning attribute ID does not have duplicates; hence it suffices to add only the index key *name* and the attribute ID to the *index_name* relation. Otherwise, further attributes would have to be added to ensure tuples can be uniquely identified, as described next.

In general, given a relation r, which is partitioned on a set of attributes K_p, if we wish to create a global secondary index on a set of attributes K_i, we create a new relation r_i^s, containing the following attributes:

1. K_i, and K_p
2. If the partitioning attributes K_p have duplicates, we would have to add further attributes K_u, such that $K_p \cup K_u$ is a key for the relation being indexed.

The relation r_i^s is partitioned on K_i, and a local index is created on K_i. In addition, a local index on attributes (K_p, K_u) is created on each partition of relation r.

We now consider how to use a global secondary index to answer a query. Consider a query that specifies a particular value v for K_i. The query is processed as follows:

1. The relevant partition of r_i^s for the value v is found using the partitioning function on K_i.
2. Use the local index on K_i at the above partition to find tuples of r_i^s that have the specified value v for K_i.
3. The tuples in the preceding result are partitioned based on the K_p value and sent to the corresponding nodes.
4. At each node, the tuples received from the preceding step are used along with the local index on r on attributes $K_p \cup K_u$, to find matching r tuples.

Note that relation r_i^s is basically a materialized view defined as $r_i^s = \Pi_{K_i,K_p,K_u}(r)$. Whenever r is modified by inserts, deletions, or updates to tuples, the materialized view r_i^s must be correspondingly updated.

Note also that updates to a tuple of r at some node may result in updates to tuples of r_i^s at other nodes. For example, in Figure 21.6, if the name of ID 001 is updated from Zhang to Yang, the tuple (Zhang, 001) at Tablet8 will be updated to (Yang, 001); since both Zhang and Yang belong in the same partition of the secondary index, no other partition is affected. On the other hand, if the name is updated from Zhang to Bolin, the tuple (Zhang, 001) will be deleted from Tablet8 and a new entry (Bolin, 001) added to Tablet6.

Performing the above updates to the secondary index as part of the same transaction that updates the base relation requires updates to be committed atomically across multiple nodes. Two-phase commit, discussed in Section 23.2, can be used for this task. Alternatives based on persistent messaging can also be used as described in Section 23.2.3, provided it is acceptable for the secondary index to be somewhat out of date.

21.6 Distributed File Systems

A **distributed file system** stores files across a large collection of machines while giving a single-file-system view to clients. As with any file system, there is a system of file names and directories, which clients can use to identify and access files. Clients do not need to bother about where the files are stored.

The goal of first-generation distributed file systems was to allow client machines to access files stored on one or more file servers. In contrast, later-generation distributed file systems, which we focus on, address distribution of file blocks across a very large number of nodes. Such distributed file systems can store very large amounts of data and support very large numbers of concurrent clients. A landmark system in this context was the Google File System (GFS), developed in the early 2000s, which saw widespread use within Google. The open-source Hadoop File System (HDFS) is based on the GFS architecture and is now very widely used.

Distributed file systems are generally designed to efficiently store large files whose sizes range from tens of megabytes to hundreds of gigabytes or more. However, they are designed to store moderate numbers of such files, of the order of millions; they are typically not designed to stores billions of different files. In contrast, the parallel data storage systems we have seen earlier are designed to store very large numbers (billions or more) of data items, whose size can range from small (tens of bytes) to medium (a few megabytes).

As in parallel data storage systems, the data in a distributed file system are stored across a number of nodes. Since files can be much larger than data items in a data storage system, files are broken up into multiple blocks. The blocks of a single file can be partitioned across multiple machines. Further, each file block is replicated across multiple (typically three) machines, so that a machine failure does not result in the file becoming inaccessible.

File systems typically support two kinds of *metadata*:

1. A directory system, which allows a hierarchical organization of files into directories and subdirectories, and
2. A mapping from a file name to the sequence of identifiers of blocks that store the actual data in each file.

In the case of a centralized file system, the block identifiers help locate blocks in a storage device such as a disk. In the case of a distributed file system, in addition to providing a block identifier, the file system must provide the location (node identifier) where the block is stored; in fact, due to replication, the file system provides a set of node identifiers along with each block identifier.

In the rest of this section, we describe the organization of the Hadoop File System (HDFS), which is shown in Figure 21.7; the architecture of HDFS is derived from that of the Google File System (GFS). The nodes (machines) which store data blocks in HDFS are called **datanodes**. Blocks have an associated ID, and datanodes map the block ID to a location in their local file system where the block is stored.

The file system metadata too can be partitioned across many nodes, but unless carefully architected, this could lead to bad performance. GFS and HDFS took a simpler and more pragmatic approach of storing the file system metadata at a single node, called the **namenode** in HDFS.

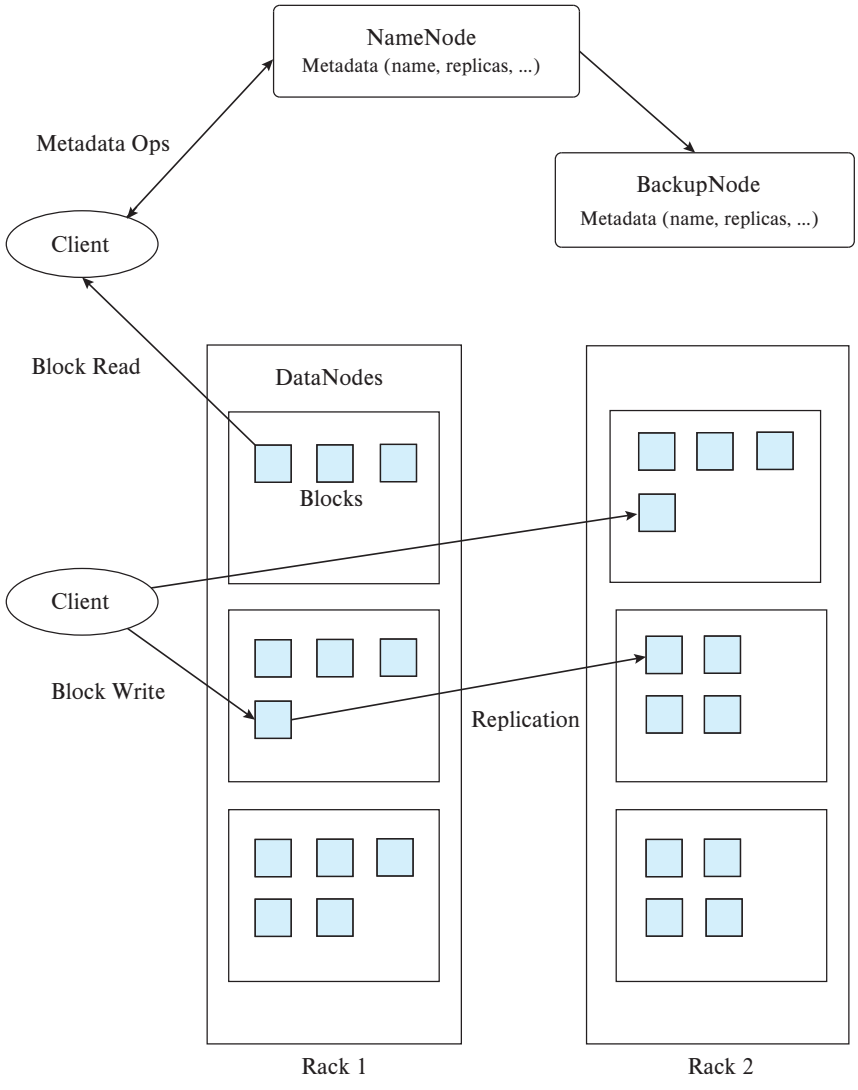

Figure 21.7 Hadoop Distributed File System (HDFS) architecture

Since all metadata reads have to go to the namenode, if a disk access were required to satisfy a metadata read, the number of requests that could be satisfied per second would be very small. To ensure acceptable performance, HDFS namenodes cache the entire metadata in memory; the size of memory then becomes a limiting factor in the number of files and blocks that the file system can manage. To reduce the memory size, HDFS uses very large block sizes (typically 64 MB) to reduce the number of blocks that the namenode must track for each file. Despite this, the limited amount of main memory on most machines constrains namenodes to support only a limited number of

files (of the order of millions). However, with main-memory sizes of many gigabytes, and block sizes of tens of megabytes, an HDFS system can comfortably handle many petabytes of data.

In any system with a large number of datanodes, datanode failures are a frequent occurrence. To deal with datanode failures, data blocks must be replicated to multiple datanodes. If a datanode fails, the block can still be read from one of the other datanodes that stores a replica of the block. Replication to three datanodes is widely used to provide high availability, without paying too high a storage overhead.

We now consider how a file open and read request is satisfied with HDFS. First, the client contacts the namenode, with the name of the file. The namenode finds the list of IDs of blocks containing the file data and returns to the client the list of block IDs, along with the set of nodes that contain replicas of each of the blocks. The client then contacts any one of the replicas for each block of the file, sending it the ID of the block, to retrieve the block. In case that particular replica does not respond, the client can contact any of the other replicas.

To satisfy a write request, the client first contacts the namenode, which allocates blocks, and decides which datanodes should store replicas of each block. The metadata are recorded at the namenode and sent back to the client. The client then writes the block to all the replicas. As an optimization to reduce network traffic, HDFS implementations may choose to store two replicas in the same rack; in that case, the block write is performed to one replica, which then copies the data to the second replica on the same rack. When all the replicas have processed the write of a block, an acknowledgment is sent to the client.

Replication introduces the problem of consistency of data across the replicas in case the file is updated. As an example, suppose one of the replicas of a data block is updated, but due to system failure, another replica does not get updated; then the system could end up with inconsistent states across the replicas. And what value is read would depend on which replica is accessed, which is not an acceptable situation.

While data storage systems in general need to deal with consistency, using techniques that we study in Chapter 23, some distributed file systems such as HDFS take a different approach: namely, not allowing updates. In other words, a file can be appended to, but data that are written cannot be updated. As each block of the file is written, the block is copied to the replicas. The file cannot be read until it is *closed*, that is, all data have been written to the file, and the blocks have been written successfully at all their replicas. The model of writing data to a file is sometimes referred to as **write-once-read-many** access model. Others such as GFS allow updates and detect certain inconsistent states caused by failures while writing to replicas; however, transactional (atomic) updates are not supported.

The restriction that files cannot be updated, but can only be appended to, is not a problem for many applications of a distributed file system. Applications that require updates should use a data-storage system that supports updates instead of using a distributed file system.

21.7 Parallel Key-Value Stores

Many Web applications need to store very large numbers (many billions) of relatively small records (of size ranging from a few kilobytes to a few megabytes). Storage would have to be distributed across thousands of nodes. Storing such records as files in a distributed file system is infeasible, since file systems are not designed to store such large numbers of small files. Ideally, a massively parallel relational database should be used to store such data. But the parallel relational databases available in the early 2000s were not designed to work at a massive scale; nor did they support the ability to easily add more nodes to the system without causing significant disruption to ongoing activities.

A number of parallel key-value storage systems were developed to meet the needs of such web applications. A **key-value store** provides a way to store or update a data item (value) with an associated key and to retrieve the data item with a given key. Some key-value stores treat the data items as uninterpreted sequences of bytes, while others allow a schema to be associated with the data item. If the system supports definition of a schema for data items, it is possible for the system to create and maintain secondary indices on specified attributes of data items.

Key-value stores support two very basic functions on tables: `put(table, key, value)`, used to store values, with an associated key, in a table, and `get(table, key)`, which retrieves the stored value associated with the specified key. In addition, they may support other functions, such as range queries on key values, using `get(table, key1, key2)`.

Further, many key-value stores support some form of flexible schema.

- Some allow column names to be specified as part of a schema definition, similar to relational data stores.
- Others allow columns to be added to, or deleted from, individual tuples; such key-value stores are sometimes referred to as **wide-column stores**. Such key-value stores support functions such as `put(table, key, columnname, value)`, to store a value in a specific column of a row identified by the key (creating the column if it does not already exist), and `get(table, key, columnname)`, which retrieves the value for a specific column of a specific row identified by the key. Further, `delete(table, key, columnname)` deletes a specific column from a row.
- Yet other key-value stores allow the value stored with a key to have a complex structure, typically based on JSON; they are sometimes referred to as **document stores**.

The ability to specify a (partial) schema of the stored value allows the key-value store to evaluate selection predicates at the data store; some stores also use the schema to support secondary indices.

We use the term *key-value store* to include all the above types of data stores; however, some people use the term *key-value store* to refer more specifically to those that

do not support any form of schema and treat the value as an uninterpreted sequence of bytes.

Parallel key-value stores typically support *elasticity*, whereby the number of nodes can be increased or decreased incrementally, depending on demand. As nodes are added, tablets can be moved to the new nodes. To reduce the number of nodes, tablets can be moved away from some nodes, which can then be removed from the system.

Widely used parallel key-value stores that support flexible columns (also known as wide-column stores) include Bigtable from Google, Apache HBase, Apache Cassandra (originally developed at Facebook), and Microsoft Azure Table Storage from Microsoft, among others. Key-value stores that support a schema include Megastore and Spanner from Google, and Sherpa/PNUTS from Yahoo!. Key-value stores that support semi-structured data (also known as document-stores) include Couchbase, DynamoDB from Amazon, and MongoDB, among others. Redis and Memcached are parallel in-memory key-value stores which are widely used for caching data.

Key-value stores are not full-fledged databases, since they do not provide many of the features that are viewed as standard on database systems today. Features that key-value stores typically do not support include declarative querying (using SQL or any other declarative query language), support for transactions, and support for efficient retrieval of records based on selections on nonkey attributes (traditional databases support such retrieval using secondary indices). In fact, they typically do not support primary-key constraints for attributes other than the key, and do not support foreign-key constraints.

21.7.1 Data Representation

As an example of data management needs of web applications, consider the profile of a user, which needs to be accessible to a number of different applications that are run by an organization. The profile contains a variety of attributes, and there are frequent additions to the attributes stored in the profile. Some attributes may contain complex data. A simple relational representation is often not sufficient for such complex data.

Many key-value stores support the *JavaScript Object Notation* (*JSON*) representation, which has found increasing acceptance for representing complex data (JSON is covered in Section 8.1.2). The JSON representation provides flexibility in the set of attributes that a record contains, as well as the types of these attributes. Yet others, such as Bigtable, define their own data model for complex data, including support for records with a very large number of optional columns.

In Bigtable, a record is not stored as a single value but is instead split into component attributes that are stored separately. Thus, the key for an attribute value conceptually consists of (record-identifier, attribute-name). Each attribute value is just a string as far as Bigtable is concerned. To fetch all attributes of a record, a range query, or more precisely a prefix-match query consisting of just the record identifier, is used. The `get()` function returns the attribute names along with the values. For efficient retrieval

of all attributes of a record, the storage system stores entries sorted by the key, so all attribute values of a particular record are clustered together.

In fact, the record identifier can itself be structured hierarchically, although to Bigtable itself the record identifier is just a string. For example, an application that stores pages retrieved from a web crawl could map a URL of the form:

www.cs.yale.edu/people/silberschatz.html

to the record identifier:

edu.yale.cs.www/people/silberschatz.html

so that pages are clustered in a useful order.

Data-storage systems often allow multiple versions of data items to be stored. Versions are often identified by timestamp, but they may be alternatively identified by an integer value that is incremented whenever a new version of a data item is created. Reads can specify the required version of a data item, or they can pick the version with the highest version number. In Bigtable, for example, a key actually consists of three parts: (record-identifier, attribute-name, timestamp).

Some key-value stores support *columnar storage* of rows, with each column of a row stored separately, with the row key and the column value stored for each row. Such a representation allows a scan to efficiently retrieve a specified column of all rows, without having to retrieve other columns from storage. In contrast, if rows are stored in the usual manner, with all column values stored with the row, a sequential scan of the storage would fetch columns that are not required, reducing performance.

Further, some key-value stores support the notion of a **column family**, which groups sets of columns into a column family. For a given row, all the columns in a specific column family are stored together, but columns from other column families are stored separately. If a set of columns are often retrieved together, storing them as a column family may allow more efficient retrieval, as compared to either columnar storage where these are stored and retrieved separately, or a row storage, which could result in retrieving unneeded columns from storage.

21.7.2 Storing and Retrieving Data

In this section, we use the term *tablet* to refer to partitions, as discussed in Section 21.3.3. We also use the term **tablet server** to refer to the node that acts as the server for a particular tablet; all requests related to a tablet are sent to the tablet server for that tablet.[5]. The tablet server would be one of the nodes that has a replica of the tablet and plays the role of master replica as discussed in Section 21.4.1.[6]

[5] HBase uses the terms *region* and *region server* in place of the terms *tablet* and *tablet server*

[6] In BigTable and HBase, replication is handled by the underlying distributed file system; tablet data are stored in files, and one of the nodes containing a replica of the tablet files is chosen as the tablet server.

We use the term **master** to refer to a site that stores a master copy of the partition information, including, for each tablet, the key ranges for the tablet, the sites storing the replicas of the tablet, and the current tablet server for that tablet.[7] The master is also responsible for tracking the health of tablet servers; in case a tablet server node fails, the master assigns one of the other nodes that contains a replica of the tablet to act as the new tablet server for that tablet. The master is also responsible for reassigning tablets to balance the load in the system if some node is overloaded or if a new node is added to the system.

For each request coming into the system, the tablet corresponding to the key must be identified, and the request routed to the tablet server. If a single master site were responsible for this task, it would get overloaded. Instead, the routing task is parallelized in one of two ways:

- By replicating the partition information to the client sites; the key-value store API used by clients looks up the partition information copy stored at the client to decide where to route a request. This approach is used in Bigtable and HBase.
- By replicating the partition information to a set of router sites, which route requests to the site with the appropriate tablet. Requests can be sent to any one of the router sites, which forward the request to the correct tablet master. This approach is used, for example, in the PNUTS system.

Since there may be a gap between actually splitting or moving a tablet and updating the partition information at a router (or client), the partition information may be out of date when the routing decision is made. When the request reaches the identified tablet master node, the node detects that the tablet has been split, or that the site no longer stores a (master) replica of the tablet. In such a case, the request is returned to the router with an indication that the routing was incorrect; the router then retrieves up-to-date tablet mapping information from the master and reroutes the request to the correct destination.

Figure 21.8 depicts the architecture of a cloud data-storage system, based loosely on the PNUTS architecture. Other systems provide similar functionality, although their architecture may vary. For example, Bigtable/HBase do not have separate routers; the partitioning and tablet-server mapping information is stored in the Google File System/HDFS, and clients read the information from the file system and decide where to send their requests.

21.7.2.1 Geographically Distributed Storage

Several key-value stores support replication of data to geographically distributed locations; some of these also support partitioning of data across geographically distributed locations, allowing different partitions to be replicated in different sets of locations.

[7] The term *tablet controller* is used by PNUTS to refer to the master site.

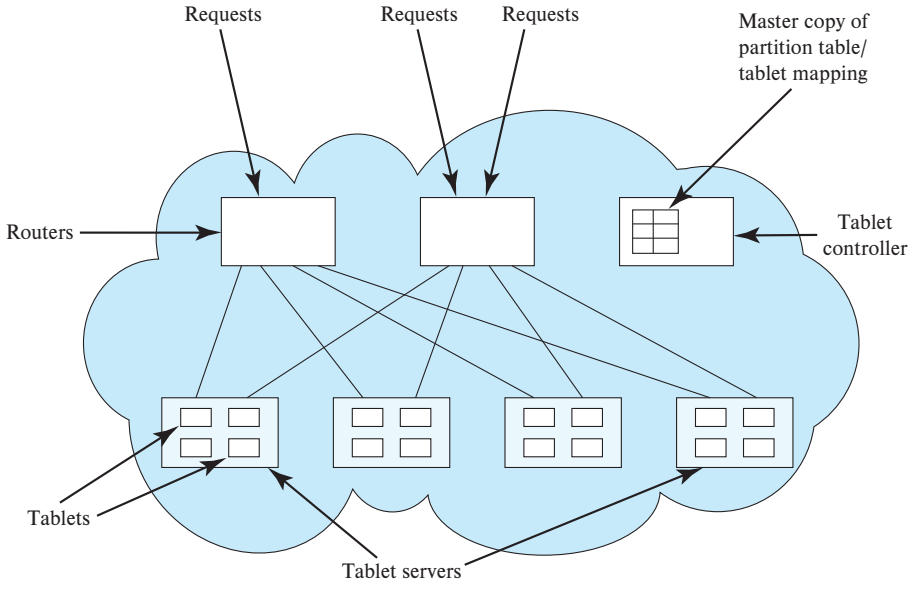

Figure 21.8 Architecture of a cloud data storage system.

One of the key motivations for geographic distribution is fault tolerance, which allows the system to continue functioning even if an entire data center fails due to a disaster such as a fire or an earthquake; in fact, earthquakes could cause all data centers in a region to fail. A second key motivation is to allow a copy of the data to reside at a geographic region close to the user; requiring data to be fetched from across the world could result in latencies of hundreds of milliseconds.

A key performance issue with geographical replication of data is that the latency across geographical regions is much higher than the latency within a data center. Some key-value stores nevertheless support geographically distributed replication, requiring transactions to wait for confirmation of updates from remote locations. Other key-value stores support asynchronous replication of updates to remote locations, allowing a transaction to commit without waiting for confirmation of updates from a remote location. There is, however, a risk of loss of updates in case of failure before the updates are replicated. Some key-value stores allow the application to choose whether to wait for confirmation from remote locations or to commit as soon as updates are performed locally.

Key-value stores that support geographic replication include Apache Cassandra, Megastore and Spanner from Google, Windows Azure storage from Microsoft, and PNUTS/Sherpa from Yahoo!, among others.

21.7.2.2 Index Structure

The records in each tablet in a key-value store are indexed on the key; range queries can be efficiently supported by storing records clustered on the key. A B^+-tree file organization is a good option, since it supports indexing with clustered storage of records.

The widely used key-value stores BigTable and HBase are built on top of distributed file systems in which files are *immutable*; that is, files cannot be updated once they are created. Thus B^+-tree indices or file organization cannot be stored in immutable files, since B^+-trees require updates, which cannot be done on an immutable file.

Instead, the BigTable and HBase systems use the *stepped-merge* variant of the *log structured merge tree* (*LSM tree*), which we saw in Section 14.8.1, and is described in more detail in Section 24.2. The LSM tree does not perform updates on existing trees, but instead creates new trees either using new data or by merging existing trees. Thus, it is an ideal fit for use on top of distributed file systems that only support immutable files. As an extra benefit, the LSM tree supports clustered storage of records, and can support very high insert and update rates, which has been found very useful in many applications of key-value stores. Several key-value stores, such as Apache Cassandra and the WiredTiger storage structure used by MongoDB, use the LSM tree structure.

21.7.3 Support for Transactions

Most key-value stores offer limited support for transactions. For example, key-value stores typically support atomic updates on a single data item and ensure that updates on the data item are serialized, that is, run one after the other. Serializability at the level of individual operations is thus trivially satisfied, since the operations are run serially. Note that serializability at the level of transactions is not guaranteed by serial execution of updates on individual data items, since a transaction may access more than one data item.

Some key-value stores, such as Google's MegaStore and Spanner, provide full support for ACID transactions across multiple nodes. However, most key-value stores do not support transactions across multiple data items.

Some key-value stores provide a test-and-set operation that can help applications implement limited forms of concurrency control, as we see next.

21.7.3.1 Concurrency Control

Some key-value stores, such as the Megastore and Spanner systems from Google, support concurrency control via locking. Issues in distributed concurrency control are discussed in Chapter 23. Spanner also supports versioning and database snapshots based on timestamps. Details of the multiversion concurrency control technique implemented in Spanner are discussed in Section 23.5.1.

However, most of the other key-value stores, such as Bigtable, PNUTS/Sherpa, and MongoDB, support atomic operations on single data items (which may have multiple columns, or may be JSON documents in MongoDB).

Some key-value stores, such as HBase and PNUTS, provide an atomic test-and-set function, which allows an update to a data item to be conditional on the current version of the data item being the same as a specified version number; the check (test) and the update (set) are performed atomically. This feature can be used to implement a limited form of validation-based concurrency control, as discussed in Section 23.3.7.

Some data stores support atomic increment operations on data items and atomic execution of stored procedures. For example, HBase supports the incrementColumnValue() operation, which atomically reads and increments a column value, and a checkAndPut() which atomically checks a condition on a data item and updates it only if the check succeeds. HBase also supports atomic execution of stored procedures, which are called "coprocessors" in HBase terminology. These procedures run on a single data item and are executed atomically.

21.7.3.2 Atomic Commit

BigTable, HBase, and PNUTS support atomic commit of multiple updates to a single row; however, none of these systems supports atomic updates across different rows.

As one of the results of the above limitation, none of these systems supports secondary indices; updates to a data item would require updates to the secondary index, which cannot be done atomically.

Some systems, such as PNUTS, support secondary indices or materialized views with deferred updates; updates to a data item result in updates to the secondary index or materialized view being added to a messaging service to be delivered to the node where the update needs to be applied. These updates are guaranteed to be delivered and applied subsequently; however, until they are applied, the secondary index may be inconsistent with the underlying data. View maintenance is also supported by PNUTS in the same deferred fashion. There is no transactional guarantee on the updates of such secondary indices or materialized views, and only a best-effort guarantee in terms of when the updates reach their destination. Consistency issues with deferred maintenance are discussed in Section 23.6.3.

In contrast, the Megastore and Spanner systems developed by Google support atomic commit for transactions spanning multiple data items, which can be spread across multiple nodes. These systems use two-phase commit (discussed in Section 23.2) to ensure atomic commit across multiple nodes.

21.7.3.3 Dealing with Failures

If a tablet server node fails, another node that has a copy of the tablet should be assigned the task of serving the tablet. The master node is responsible for detecting node failures and reassigning tablet servers.

When a new node takes over as tablet server, it must recover the state of the tablet. To ensure that updates to the tablet survive node failures, updates to a tablet are logged, and the log is itself replicated. When a site fails, the tablets at the site are assigned to

other sites; the new master site of each tablet is responsible for performing recovery actions using the log to bring its copy of the tablet to an up-to-date state, after which updates and reads can be performed on the tablet.

In Bigtable, as an example, mapping information is stored in an index structure, and the index, as well as the actual tablet data, are stored in the file system. Tablet data updates are not flushed immediately, but log data are. The file system ensures that the file system data are replicated and will be available even in the face of failure of a few nodes in the cluster. Thus, when a tablet is reassigned, the new server for that tablet has access to up-to-date log data.

Yahoo!'s Sherpa/PNUTS system, on the other hand, explicitly replicates tablets to multiple nodes in a cluster and uses a persistent messaging system to implement the log. The persistent messaging system replicates log records at multiple sites to ensure availability in the event of a failure. When a new node takes over as the tablet server, it must apply any pending log records that were generated by the earlier tablet server before taking over as the tablet server.

To ensure availability in the face of failures, data must be replicated. As noted in Section 21.4.2, a key issue with replication is the task of keeping the replicas consistent with each other. Different systems implement atomic update of replicas in different fashions. Google BigTable and Apache HBase use replication features provided by an underlying file system (GFS for BigTable, and HDFS for HBase), instead of implementing replication on their own. Interestingly, neither GFS nor HDFS supports atomic updates of all replicas of a file; instead they support appends to files, which are copied to all replicas of the file blocks. An append is successful only when it has been applied to all replicas. System failures can result in appends that are applied to only some replicas; such incomplete appends are detected using sequence numbers and are cleaned up when they are detected.

Some systems such as PNUTS use a persistent messaging service to log updates; the messaging service guarantees that updates will be delivered to all replicas. Other systems, such as Google's Megastore and Spanner, use a technique called distributed consensus to implement consistent replication, as we discuss in Section 23.8. Such systems require a majority of replicas to be available to perform an update. Other systems, such as Apache Cassandra and MongoDB, allow the user control over how many replicas must be available to perform an update. Setting the value low could result in conflicting updates, which must be resolved later. We discuss these issues in Section 23.6.

21.7.4 Managing Without Declarative Queries

Key-value stores do not provide any query processing facility, such as SQL language support, or even lower-level primitives such as joins. Many applications that use key-value stores can manage without query language support. The primary mode of data access in such applications is to store data with an associated key and to retrieve data

with that key. In the user profile example, the key for user-profile data would be the user's identifier.

There are applications that require joins but implement the joins either in application code or by a form of view materialization. For example, in a social-networking application, each user should be shown new posts from all her friends, which conceptually requires a join.

One approach to computing the join is to implement it in the application code, by first finding the set of friends of a given user, and then querying the data object representing each friend, to find their recent posts.

An alternative approach is as follows: Whenever a user makes a post, for each friend of the user a message is sent to, the data object representing that friend and the data associated with the friend are updated with a summary of the new post. When that user checks for updates, all required data are available in one place and can be retrieved quickly.

Both approaches can be used without any underlying support for joins. There are trade-offs between the two alternatives such as higher cost at query time for the first alternative versus higher storage cost and higher cost at the time of writes for the second alternative.

21.7.5 Performance Optimizations

When using a data storage system, the physical location of data are decided by the storage system and hidden from the client. When storing multiple relations that need to be joined, partitioning each independently may be suboptimal in terms of communication cost. For example, if the join of two relations is computed frequently, it may be best if they are partitioned in exactly the same way, on their join attributes. As we will see in Section 22.7.4, doing so would allow the join to be computed in parallel at each storage site, without data transfer.

To support such scenarios, some data storage systems allow the schema designer to specify that tuples of one relation should be stored in the same partitions as tuples of another relation that they reference, typically using a foreign key. A typical use of this functionality is to store all tuples related to a particular entity together in the same partition; the set of such tuples is called an **entity group**.

Further, many data storage systems, such as HBase, support *stored functions* or *stored procedures*. Stored functions/procedures allow clients to invoke a function on a tuple (or an entity group) and instead of the tuples being fetched and executed locally, the function is executed at the partition where the tuple is stored. Stored functions/procedures are particularly useful if the stored tuples are large, while the function/procedure results are small, reducing data transfer.

Many data storage systems provide features such as support for automatically deleting old versions of data items after some period of time, or even deleting data items that are older than some specified period.

21.8 Summary

- Parallel databases have gained significant commercial acceptance in the past 20 years.
- Data storage and indexing are two important aspects of parallel database systems.
- Data partitioning involves the distribution of data among multiple nodes. In I/O parallelism, relations are partitioned among available disks so that they can be retrieved faster. Three commonly used partitioning techniques are round-robin partitioning, hash partitioning, and range partitioning.
- Skew is a major problem, especially with increasing degrees of parallelism. Balanced partitioning vectors, using histograms, and virtual node partitioning are among the techniques used to reduce skew.
- Parallel data storage systems must be resilient to failure of nodes. To ensure that data are not lost on node failure, tuples are replicated across at least two nodes, and often three nodes. If a node fails, the tuples that it stored can still be accessed from the other nodes where the tuples are replicated.
- Indices in a parallel data storage system can be divided into two kinds: local and global. A local index is an index built on tuples stored in a particular node; The index contents are stored on the same node as the data. A global index is an index built on data stored across multiple nodes.
- A distributed file system stores files across a large collection of machines, while giving a single-file-system view to clients. As with any file system, there is a system of file names and directories, which clients can use to identify and access files. Clients do not need to bother about where the files are stored.
- Web applications need to store very large numbers (many billions) of relatively small records (of size ranging from a few kilobytes to a few megabytes). A number of parallel key-value storage systems were developed to meet the needs of such web applications. A key-value store provides a way to store or update a data item (value) with an associated key, and to retrieve the data item with a given key.

Review Terms

- Key-value stores
- Data storage system
- I/O parallelism
- Data partitioning
- Horizontal partitioning
- Partitioning strategies
  - Round-robin
  - Hash partitioning
  - Range partitioning
- Partitioning vector

heterogeneous distributed databases. Replication with weak degrees of consistency is discussed in Section 23.6.

Most techniques for dealing with distributed data require the use of coordinators to ensure consistent and efficient transaction processing. In Section 23.7 we discuss how coordinators can be chosen in a distributed fashion, robust to failures. Finally, Section 23.8 describes the distributed consensus problem, outlines solutions for the problem, and then discusses how these solutions can be used to implement fault-tolerant services by means of replication of a log.

23.1 Distributed Transactions

Access to the various data items in a distributed system is usually accomplished through transactions, which must preserve the ACID properties (Section 17.1). There are two types of transaction that we need to consider. The **local transactions** are those that access and update data in only one local database; the **global transactions** are those that access and update data in several local databases. Ensuring the ACID properties of the local transactions can be done as described in Chapter 17, Chapter 18, and Chapter 19. However, for global transactions, this task is much more complicated, since several nodes may be participating in the execution of the transaction. The failure of one of these nodes, or the failure of a communication link connecting these nodes, may result in erroneous computations.

In this section, we study the system structure of a distributed database and its possible failure modes. In later sections, we discuss how to ensure ACID properties are satisfied in a distributed database, despite failures. We reemphasize that these failure modes occur with parallel databases as well, and the techniques we describe are equally applicable to parallel databases.

23.1.1 System Structure

We now consider a system structure with multiple nodes, each of which can fail independently of the others. We note that the nodes may be within a single data center, corresponding to a parallel database system, or geographically distributed, in a distributed database system. The system structure is similar in either case; the problems with respect to transaction isolation and atomicity are the same in both cases, as are the solutions.

We note that the system structure we consider here is not applicable to a shared-memory parallel database system whose components do not have independent modes of failures. In such systems either the whole system is up, or the whole system is down. Further, there is usually only one transaction log used for recovery. Concurrency control and recovery techniques that are designed for centralized database systems can be used in such systems, and are preferable to techniques described in this chapter.

Each node has its own *local* transaction manager, whose function is to ensure the ACID properties of those transactions that execute at that node. The various trans-

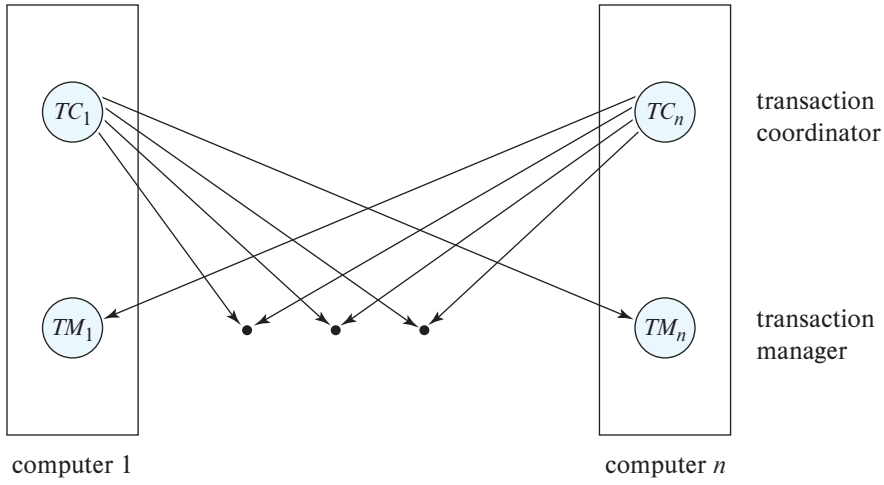

Figure 23.1 System architecture.

action managers cooperate to execute global transactions. To understand how such a manager can be implemented, consider an abstract model of a transaction system, in which each node contains two subsystems:

- The **transaction manager** manages the execution of those transactions (or subtransactions) that access data stored in the node. Note that each such transaction may be either a local transaction (i.e., a transaction that executes at only that node) or part of a global transaction (i.e., a transaction that executes at several nodes).
- The **transaction coordinator** coordinates the execution of the various transactions (both local and global) initiated at that node.

The overall system architecture appears in Figure 23.1.

The structure of a transaction manager is similar in many respects to the structure of a centralized system. Each transaction manager is responsible for:

- Maintaining a log for recovery purposes.
- Participating in an appropriate concurrency-control scheme to coordinate the concurrent execution of the transactions executing at that node.

As we shall see, we need to modify both the recovery and concurrency schemes to accommodate the distributed execution of transactions.

The transaction coordinator subsystem is not needed in the centralized environment, since a transaction accesses data at only a single node. A transaction coordinator, as its name implies, is responsible for coordinating the execution of all the transactions initiated at that node. For each such transaction, the coordinator is responsible for:

- Starting the execution of the transaction.
- Breaking the transaction into a number of subtransactions and distributing these subtransactions to the appropriate nodes for execution.
- Coordinating the termination of the transaction, which may result in the transaction being committed at all nodes or aborted at all nodes.

23.1.2 System Failure Modes

A distributed system may suffer from the same types of failure that a centralized system does (e.g., software errors, hardware errors, or disk crashes). There are, however, additional types of failure with which we need to deal in a distributed environment. The basic failure types are:

- Failure of a node.
- Loss of messages.
- Failure of a communication link.
- Network partition.

The loss or corruption of messages is always a possibility in a distributed system. The system uses transmission-control protocols, such as TCP/IP, to handle such errors. Information about such protocols may be found in standard textbooks on networking.

However, if two nodes A and B are not directly connected, messages from one to the other must be *routed* through a sequence of communication links. If a communication link fails, messages that would have been transmitted across the link must be rerouted. In some cases, it is possible to find another route through the network, so that the messages are able to reach their destination. In other nodes, a failure may result in there being no connection between some pairs of nodes. A system is **partitioned** if it has been split into two (or more) subsystems, called **partitions**, that lack any connection between them. Note that, under this definition, a partition may consist of a single node.

23.2 Commit Protocols

If we are to ensure atomicity, all the nodes in which a transaction T executed must agree on the final outcome of the execution. T must either commit at all nodes, or it must abort at all nodes. To ensure this property, the transaction coordinator of T must execute a commit protocol.

Among the simplest and most widely used commit protocols is the **two-phase commit protocol** (**2PC**), which is described in Section 23.2.1.

23.2.1 Two-Phase Commit

We first describe how the two-phase commit protocol (2PC) operates during normal operation, then describe how it handles failures and finally how it carries out recovery and concurrency control.

Consider a transaction T initiated at node N_i, where the transaction coordinator is C_i.

23.2.1.1 The Commit Protocol

When T completes its execution—that is, when all the nodes at which T has executed inform C_i that T has completed—C_i starts the 2PC protocol.

- **Phase 1**. C_i adds the record <prepare T> to the log and forces the log onto stable storage. It then sends a prepare T message to all nodes at which T executed.
  On receiving such a message, the transaction manager at that node determines whether it is willing to commit its portion of T. If the answer is no, it adds a record <no T> to the log and then responds by sending an abort T message to C_i. If the answer is yes, it adds a record <ready T> to the log and forces the log (with all the log records corresponding to T) onto stable storage. The transaction manager then replies with a ready T message to C_i.
- **Phase 2**. When C_i receives ready responses to the prepare T message from all the nodes, or when it receives an abort T message from at least one participant node, C_i can determine whether the transaction T can be committed or aborted. Transaction T can be committed if C_i received a ready T message from all the participating nodes. Otherwise, transaction T must be aborted. Depending on the verdict, either a record <commit T> or a record <abort T> is added to the log and the log is forced onto stable storage. At this point, the fate of the transaction has been sealed.

  Following this point, the coordinator sends either a commit T or an abort T message to all participating nodes. When a node receives that message, it records the result (either <commit T> or <abort T>) in its log, and correspondingly either commits or aborts the transaction.

  Since nodes may fail, the coordinator cannot wait indefinitely for responses from all the nodes. Instead, when a prespecified interval of time has elapsed since the prepare T message was sent out, if any node has not responded to the coordinator, the coordinator can decide to abort the transaction; the steps described for aborting the transaction must be followed, just as if a node had sent an abort message for the transaction.

Figure 23.2 shows an instance of successful execution of 2PC for a transaction T, with two nodes, N_1 and N_2, that are both willing to commit transaction T. If any of the

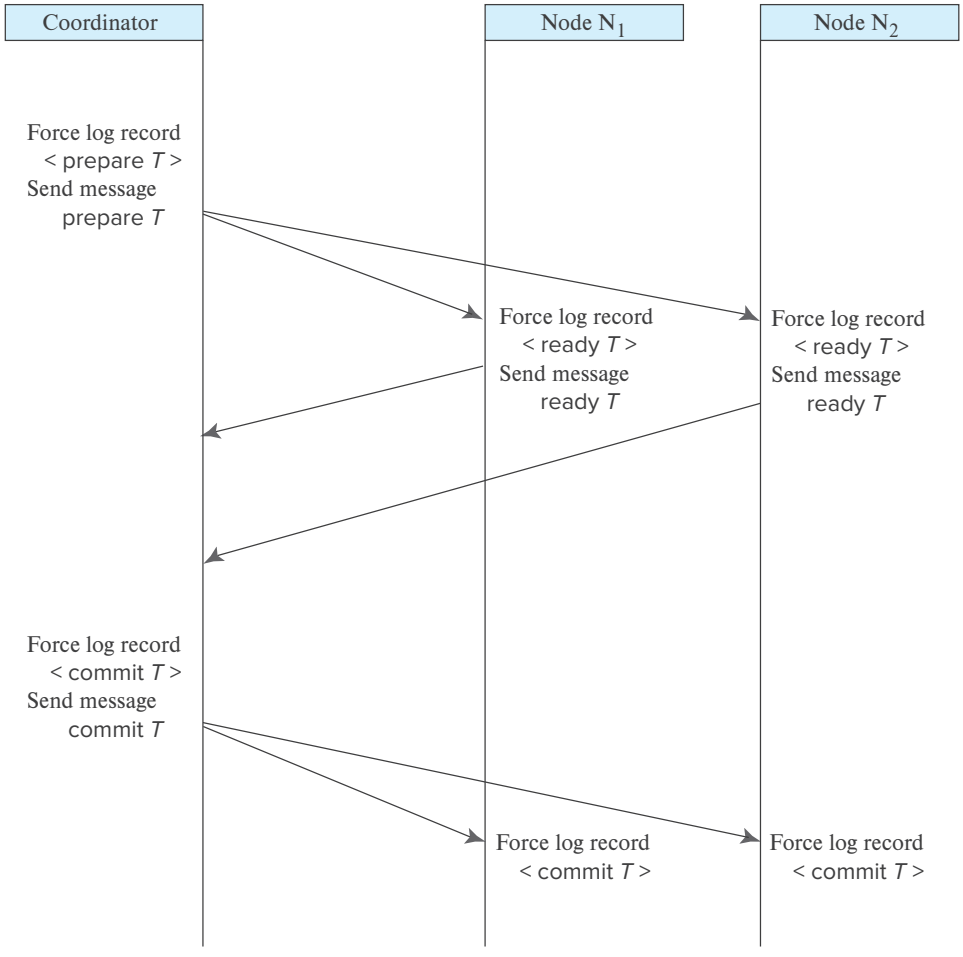

Figure 23.2 Successful execution of 2PC.

nodes sends a no T message, the coordinator will send an abort T message to all the nodes, which will then abort the transaction.

A node at which T executed can unconditionally abort T at any time before it sends the message ready T to the coordinator. Once the <ready T> log record is written, the transaction T is said to be in the **ready state** at the node. The ready T message is, in effect, a promise by a node to follow the coordinator's order to commit T or to abort T. To make such a promise, the needed information must first be stored in stable storage. Otherwise, if the node crashes after sending ready T, it may be unable to make good on its promise. Further, locks acquired by the transaction must continue to be held until the transaction completes, even if there is an intervening node failure, as we shall see in Section 23.2.1.3.

Since unanimity is required to commit a transaction, the fate of T is sealed as soon as at least one node responds abort T. Since the coordinator node N_i is one of the nodes at which T executed, the coordinator can decide unilaterally to abort T. The final verdict regarding T is determined at the time that the coordinator writes that verdict (commit or abort) to the log and forces that verdict to stable storage.

In some implementations of the 2PC protocol, a node sends an acknowledge T message to the coordinator at the end of the second phase of the protocol. When the coordinator receives the acknowledge T message from all the nodes, it adds the record <complete T> to the log. Until this step, the coordinator cannot forget about the commit or abort decision on T, since a node may ask for the decision. (A node that has not received a commit or abort for transaction T, perhaps due to a network failure or temporary node failure, may send such a request to the coordinator.) After this step, the coordinator can discard information about transaction T.

23.2.1.2 Handling of Failures

The 2PC protocol responds in different ways to various types of failure:

- **Failure of a participating node**. If the coordinator C_i detects that a node has failed, it takes these actions: If the node fails before responding with a ready T message to C_i, the coordinator assumes that it responded with an abort T message. If the node fails after the coordinator has received the ready T message from the node, the coordinator executes the rest of the commit protocol in the normal fashion, ignoring the failure of the node.

  When a participating node N_k recovers from a failure, it must examine its log to determine the fate of those transactions that were in the midst of execution when the failure occurred. Let T be one such transaction. We consider each of the possible cases:

  - The log contains a <commit T> record. In this case, the node executes redo(T).
  - The log contains an <abort T> record. In this case, the node executes undo(T).
  - The log contains a <ready T> record. In this case, the node must consult C_i to determine the fate of T. If C_i is up, it notifies N_k regarding whether T committed or aborted. In the former case, it executes redo(T); in the latter case, it executes undo(T). If C_i is down, N_k must try to find the fate of T from other nodes. It does so by sending a querystatus T message to all the nodes in the system. On receiving such a message, a node must consult its log to determine whether T has executed there, and if T has, whether T committed or aborted. It then notifies N_k about this outcome. If no node has the appropriate information (i.e., whether T committed or aborted), then N_k can neither abort nor commit T. The decision concerning T is postponed until N_k can obtain the needed

information. Thus, N_k must periodically resend the querystatus message to the other nodes. It continues to do so until a node that contains the needed information has recovered. Note that the node at which C_i resides always has the needed information.

- The log contains no control records (abort, commit, ready) concerning T. Thus, we know that N_k failed before responding to the prepare T message from C_i. Since the failure of N_k precludes the sending of such a response, by our algorithm C_i must abort T. Hence, N_k must execute undo(T).

- **Failure of the coordinator**. If the coordinator fails in the midst of the execution of the commit protocol for transaction T, then the participating nodes must decide the fate of T. We shall see that, in certain cases, the participating nodes cannot decide whether to commit or abort T, and therefore these nodes must wait for the recovery of the failed coordinator.

  - If an active node contains a <commit T> record in its log, then T must be committed.
  - If an active node contains an <abort T> record in its log, then T must be aborted.
  - If some active node does *not* contain a <ready T> record in its log, then the failed coordinator C_i cannot have decided to commit T, because a node that does not have a <ready T> record in its log cannot have sent a ready T message to C_i. However, the coordinator may have decided to abort T, but not to commit T. Rather than wait for C_i to recover, it is preferable to abort T.
  - If none of the preceding cases holds, then all active nodes must have a <ready T> record in their logs, but no additional control records (such as <abort T> or <commit T>). Since the coordinator has failed, it is impossible to determine whether a decision has been made, and if one has, what that decision is, until the coordinator recovers. Thus, the active nodes must wait for C_i to recover.

    Since the fate of T remains in doubt, T may continue to hold system resources. For example, if locking is used, T may hold locks on data at active nodes. Such a situation is undesirable, because it may be hours or days before C_i is again active. During this time, other transactions may be forced to wait for T. As a result, data items may be unavailable not only on the failed node (C_i), but on active nodes as well. This situation is called the **blocking problem**, because T is blocked pending the recovery of node C_i.

- **Network partition**. When a network partition occurs, two possibilities exist:

  1. The coordinator and all its participants remain in one partition. In this case, the failure has no effect on the commit protocol.

2. The coordinator and its participants belong to several partitions. From the viewpoint of the nodes in one of the partitions, it appears that the nodes in other partitions have failed. Nodes that are not in the partition containing the coordinator simply execute the protocol to deal with the failure of the coordinator. The coordinator and the nodes that are in the same partition as the coordinator follow the usual commit protocol, assuming that the nodes in the other partitions have failed.

Thus, the major disadvantage of the 2PC protocol is that coordinator failure may result in blocking, where a decision either to commit or to abort T may have to be postponed until C_i recovers. We discuss how to remove this limitation shortly, in Section 23.2.2.

23.2.1.3 Recovery and Concurrency Control

When a failed node restarts, we can perform recovery by using, for example, the recovery algorithm described in Section 19.4. To deal with distributed commit protocols, the recovery procedure must treat **in-doubt transactions** specially; in-doubt transactions are transactions for which a <ready T> log record is found, but neither a <commit T> log record nor an <abort T> log record is found. The recovering node must determine the commit–abort status of such transactions by contacting other nodes, as described in Section 23.2.1.2.

If recovery is done as just described, however, normal transaction processing at the node cannot begin until all in-doubt transactions have been committed or rolled back. Finding the status of in-doubt transactions can be slow, since multiple nodes may have to be contacted. Further, if the coordinator has failed, and no other node has information about the commit–abort status of an incomplete transaction, recovery potentially could become blocked if 2PC is used. As a result, the node performing restart recovery may remain unusable for a long period.

To circumvent this problem, recovery algorithms typically provide support for noting lock information in the log. (We are assuming here that locking is used for concurrency control.) Instead of writing a <ready T> log record, the algorithm writes a <ready T, L > log record, where L is a list of all write locks held by the transaction T when the log record is written. At recovery time, after performing local recovery actions, for every in-doubt transaction T, all the write locks noted in the <ready T, L > log record (read from the log) are reacquired.

After lock reacquisition is complete for all in-doubt transactions, transaction processing can start at the node, even before the commit–abort status of the in-doubt transactions is determined. The commit or rollback of in-doubt transactions proceeds concurrently with the execution of new transactions. Thus, node recovery is faster and never gets blocked. Note that new transactions that have a lock conflict with any write locks held by in-doubt transactions will be unable to make progress until the conflicting in-doubt transactions have been committed or rolled back.

23.2.2 Avoiding Blocking During Commit

The blocking problem of 2PC is a serious concern for system designers, since the failure of a coordinator node could lead to blocking of a transaction that has acquired locks on a frequently used data item, which in turn prevents other transactions that need to acquire a conflicting lock from completing their execution.

By involving multiple nodes in the commit decision step of 2PC, it is possible to avoid blocking as long as a majority of the nodes involved in the commit decision are alive and can communicate with each other. This is done by using the idea of fault-tolerant distributed consensus. Details of distributed consensus are discussed in detail later, in Section 23.8, but we outline the problem and sketch a solution approach below.

The **distributed consensus problem** is as follows: A set of n nodes need to agree on a decision; in this case, whether or not to commit a particular transaction. The inputs to make the decision are provided to all the nodes, and then each node votes on the decision; in the case of 2PC, the decision is on whether or not to commit a transaction. The key goal of protocols for achieving distributed consensus is that the decision should be made in such a way that all nodes will "learn" the same value for the decision (i.e., all nodes will learn that the transaction is to be committed, or all nodes will learn that the transaction is to be aborted), even if some nodes fail during the execution of the protocol, or there are network partitions. Further, the distributed consensus protocol should not block, as long as a majority of the nodes participating remain alive and can communicate with each other.

There are several protocols for distributed consensus, two of which are widely used today (Paxos and Raft). We study distributed consensus in Section 23.8. A key idea behind these protocols is the idea of a vote, which succeeds only if a majority of the participating nodes agree on a particular decision.

Given an implementation of distributed consensus, the blocking problem due to coordinator failure can be avoided as follows: Instead of the coordinator locally deciding to commit or abort a transaction, it initiates the distributed consensus protocol, requesting that the value "committed" or "aborted" be assigned to the transaction T. The request is sent to all the nodes participating in the distributed consensus, and the consensus protocol is then executed by those nodes. Since the protocol is fault tolerant, it will succeed even if some nodes fail, as long as a majority of the nodes are up and remain connected. The transaction can be declared committed by the coordinator only after the consensus protocol completes successfully.

There are two possible failure scenarios:

- *The coordinator fails at any stage before informing all participating nodes of the commit or abort status of a transaction T.*

  In this case, a new coordinator is chosen (we will see how to do so in Section 23.7). The new coordinator checks with the nodes participating in the distributed consensus to see if a decision was made, and if so informs the 2PC participants

of the decision. A majority of the nodes participating in consensus must respond, to check if a decision was made or not; the protocol will not block as long as the failed/disconnected nodes are in a minority.

If no decision was made earlier for transaction T, the new coordinator again checks with the 2PC participants to check if they are ready to commit or wish to abort the transaction, and follows the usual coordinator protocol based on their responses. As before, if no response is received from a participant, the new coordinator may choose to abort T.

- *The distributed consensus protocol fails to reach a decision.*

  Failure of the protocol can occur due to the failure of some participating nodes. It could also occur because of conflicting requests, none of which gets a majority of "votes" during the consensus protocol. For 2PC, the request normally comes from a single coordinator, so such a conflict is unlikely. However, conflicting requests can arise in rare cases if a coordinator fails after sending out a commit message, but its commit message is delivered late; meanwhile, a new 2PC coordinator makes an abort decision since it could not reach some participating nodes. Even with such a conflict, the distributed consensus protocol guarantees that only one of the commit or abort requests can succeed, even in the presence of failures. But if some nodes are down, and neither the commit nor the abort request gets a majority vote from nodes participating in the distributed consensus, it is possible for the protocol to fail to reach a decision.

  Regardless of the reason, if the distributed consensus protocol fails to reach a decision, the new coordinator just re-initiates the protocol.

Note that in the event of a network partition, a node that gets disconnected from the majority of the nodes participating in consensus may not learn about a decision, even if a decision was successfully made. Thus, transactions running at such a node may be blocked.

Failure of 2PC participants could make data unavailable, in the absence of replication. Distributed consensus can also be used to keep replicas of a data item in a consistent state, as we explain later in Section 23.8.4.

The idea of distributed consensus to make 2PC nonblocking was proposed in the 1980s; it is used, for example, in the Google Spanner distributed database system.

The **three-phase commit** (**3PC**) protocol is an extension of the two-phase commit protocol that avoids the blocking problem under certain assumptions. One variant of the protocol avoids blocking as long as network partitions do not occur, but it may lead to inconsistent decisions in the event of a network partition. Extensions of the protocol that work safely under network partitioning were developed subsequently. The idea behind these extensions is similar to the majority voting idea of distributed consensus, but the protocols are specifically tailored for the task of atomic commit.

23.2.3 Alternative Models of Transaction Processing

With two-phase commit, participating nodes agree to let the coordinator decide the fate of a transaction, and are forced to wait for the decision of the coordinator, while holding locks on updated data items. While such loss of autonomy may be acceptable within an organization, no organization would be willing to force its computers to wait, potentially for a long time, while a computer at another organization makes the decision.

In this section, we describe how to use *persistent messaging* to avoid the problem of distributed commit. To understand persistent messaging, consider how one might transfer funds between two different banks, each with its own computer. One approach is to have a transaction span the two nodes and use two-phase commit to ensure atomicity. However, the transaction may have to update the total bank balance, and blocking could have a serious impact on all other transactions at each bank, since almost all transactions at the bank would update the total bank balance.

In contrast, consider how funds transfer by a bank check occurs. The bank first deducts the amount of the check from the available balance and prints out a check. The check is then physically transferred to the other bank where it is deposited. After verifying the check, the bank increases the local balance by the amount of the check. The check constitutes a message sent between the two banks. So that funds are not lost or incorrectly increased, the check must not be lost and must not be duplicated and deposited more than once. When the bank computers are connected by a network, persistent messages provide the same service as the check (but much faster).

Persistent messages are messages that are guaranteed to be delivered to the recipient exactly once (neither less nor more), regardless of failures, if the transaction sending the message commits, and are guaranteed to not be delivered if the transaction aborts. Database recovery techniques are used to implement persistent messaging on top of the normal network channels, as we shall see shortly. In contrast, regular messages may be lost or may even be delivered multiple times in some situations.

Error handling is more complicated with persistent messaging than with two-phase commit. For instance, if the account where the check is to be deposited has been closed, the check must be sent back to the originating account and credited back there. Both nodes must, therefore, be provided with error-handling code, along with code to handle the persistent messages. In contrast, with two-phase commit, the error would be detected by the transaction, which would then never deduct the amount in the first place.

The types of exception conditions that may arise depend on the application, so it is not possible for the database system to handle exceptions automatically. The application programs that send and receive persistent messages must include code to handle exception conditions and bring the system back to a consistent state. For instance, it is not acceptable to just lose the money being transferred if the receiving account has been closed; the money must be credited back to the originating account, and if that is not possible for some reason, humans must be alerted to resolve the situation manually.

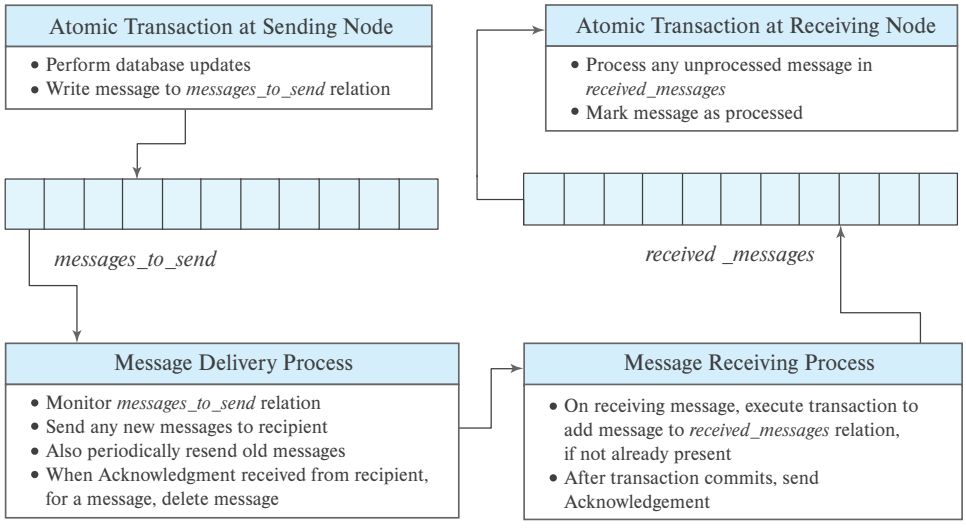

Figure 23.3 Implementation of persistent messaging.

There are many applications where the benefit of eliminating blocking is well worth the extra effort to implement systems that use persistent messages. In fact, few organizations would agree to support two-phase commit for transactions originating outside the organization, since failures could result in blocking of access to local data. Persistent messaging therefore plays an important role in carrying out transactions that cross organizational boundaries.

We now consider the **implementation** of persistent messaging. Persistent messaging can be implemented on top of an unreliable messaging infrastructure, which may lose messages or deliver them multiple times. Figure 23.3 shows a summary of the implementation, which is described in detail next.

- **Sending node protocol**. When a transaction wishes to send a persistent message, it writes a record containing the message in a special relation *messages_to_send*, instead of directly sending out the message. The message is also given a unique message identifier. Note that this relation acts as a message *outbox*.

  A *message delivery process* monitors the relation, and when a new message is found, it sends the message to its destination. The usual database concurrency-control mechanisms ensure that the system process reads the message only after the transaction that wrote the message commits; if the transaction aborts, the usual recovery mechanism would delete the message from the relation.

  The message delivery process deletes a message from the relation only after it receives an acknowledgment from the destination node. If it receives no acknowledgment from the destination node, after some time it sends the message again. It repeats this until an acknowledgment is received. In case of permanent failures,

the system will decide, after some period of time, that the message is undeliverable. Exception handling code provided by the application is then invoked to deal with the failure.

Writing the message to a relation and processing it only after the transaction has committed ensures that the message will be delivered if and only if the transaction commits. Repeatedly sending it guarantees it will be delivered even if there are (temporary) system or network failures.

- **Receiving node protocol**. When a node receives a persistent message, it runs a transaction that adds the message to a special *received_messages* relation, provided it is not already present in the relation (the unique message identifier allows duplicates to be detected). The relation has an attribute to indicate if the message has been processed, which is set to false when the message is inserted in the relation. Note that this relation acts as a message *inbox*.

  After the transaction commits, or if the message was already present in the relation, the receiving node sends an acknowledgment back to the sending node.

  Note that sending the acknowledgment before the transaction commits is not safe, since a system failure may then result in loss of the message. Checking whether the message has been received earlier is essential to avoid multiple deliveries of the message.

- **Processing of message**. Received messages must be processed to carry out the actions specified in the message. A process at the receiving node monitors the *received_messages* relation to check for messages that have not been processed. When it finds such a message, the message is processed, and as part of the same transaction that processes the message, the processed flag is set to true. This ensures that a message is processed exactly once after it is received.

- **Deletion of old messages**. In many messaging systems, it is possible for messages to get delayed arbitrarily, although such delays are very unlikely. Therefore, to be safe, the message must never be deleted from the *received_messages* relation. Deleting it could result in a duplicate delivery not being detected. But as a result, the *received_messages* relation may grow indefinitely. To deal with this problem, each message is given a timestamp, and if the timestamp of a received message is older than some cutoff, the message is discarded. All messages recorded in the *received_messages* relation that are older than the cutoff can be deleted.

Workflows provide a general model of distributed transaction processing involving multiple nodes and possibly human processing of certain steps, and they are supported by application software used by enterprises. For instance, as we saw in Section 9.6.1, when a bank receives a loan application, there are many steps it must take, including contacting external credit-checking agencies, before approving or rejecting a loan application. The steps, together, form a workflow. Persistent messaging forms the underlying basis for supporting workflows in a distributed environment.

23.3 Concurrency Control in Distributed Databases

We now consider how the concurrency-control schemes discussed in Chapter 18 can be modified so that they can be used in a distributed environment. We assume that each node participates in the execution of a commit protocol to ensure global transaction atomicity.

In this section, we assume that data items are not replicated, and we do not consider multiversion techniques. We discuss how to handle replicas later, in Section 23.4, and we discuss distributed multiversion concurrency control techniques in Section 23.5.

23.3.1 Locking Protocols

The various locking protocols described in Chapter 18 can be used in a distributed environment. We discuss implementation issues in this section.

23.3.1.1 Single Lock-Manager Approach

In the **single lock-manager** approach, the system maintains a *single* lock manager that resides in a *single* chosen node—say N_i. All lock and unlock requests are made at node N_i. When a transaction needs to lock a data item, it sends a lock request to N_i. The lock manager determines whether the lock can be granted immediately. If the lock can be granted, the lock manager sends a message to that effect to the node at which the lock request was initiated. Otherwise, the request is delayed until it can be granted, at which time a message is sent to the node at which the lock request was initiated. The transaction can read the data item from *any* one of the nodes at which a replica of the data item resides. In the case of a write, all the nodes where a replica of the data item resides must be involved in the writing.

The scheme has these advantages:

- **Simple implementation**. This scheme requires two messages for handling lock requests and one message for handling unlock requests.
- **Simple deadlock handling**. Since all lock and unlock requests are made at one node, the deadlock-handling algorithms discussed in Chapter 18 can be applied directly.

The disadvantages of the scheme are:

- **Bottleneck**. The node N_i becomes a bottleneck, since all requests must be processed there.
- **Vulnerability**. If the node N_i fails, the concurrency controller is lost. Either processing must stop, or a recovery scheme must be used so that a backup node can take over lock management from N_i, as described in Section 23.7.

23.3.1.2 Distributed Lock Manager

A compromise between the advantages and disadvantages can be achieved through the **distributed-lock-manager** approach, in which the lock-manager function is distributed over several nodes.

Each node maintains a local lock manager whose function is to administer the lock and unlock requests for those data items that are stored in that node. When a transaction wishes to lock a data item Q that resides at node N_i, a message is sent to the lock manager at node N_i requesting a lock (in a particular lock mode). If data item Q is locked in an incompatible mode, then the request is delayed until it can be granted. Once it has determined that the lock request can be granted, the lock manager sends a message back to the initiator indicating that it has granted the lock request.

The distributed-lock-manager scheme has the advantage of simple implementation, and it reduces the degree to which the coordinator is a bottleneck. It has a reasonably low overhead, requiring two message transfers for handling lock requests, and one message transfer for handling unlock requests. However, deadlock handling is more complex, since the lock and unlock requests are no longer made at a single node: There may be internode deadlocks even when there is no deadlock within a single node. The deadlock-handling algorithms discussed in Chapter 18 must be modified, as we shall discuss in Section 23.3.2, to detect global deadlocks.

23.3.2 Deadlock Handling

The deadlock-prevention and deadlock-detection algorithms in Chapter 18 can be used in a distributed system, with some modifications.

Consider first the deadlock-prevention techniques, which we saw in Section 18.2.1.

- Techniques for deadlock prevention based on lock ordering can be used in a distributed system, with no changes at all. These techniques prevent cyclic lock waits; the fact that locks may be obtained at different nodes has no effect on prevention of cyclic lock waits.
- Techniques based on preemption and transaction rollback can also be used unchanged in a distributed system. In particular, the *wait-die* technique is used in several distributed systems. Recall that this technique allows older transactions to wait for locks held by younger transactions, but if a younger transaction needs to wait for a lock held by an older transaction, the younger transaction is rolled back. The transaction that is rolled back may subsequently be executed again; recall that it retains its original start time; if it is treated as a new transaction, it could be rolled back repeatedly, and starve, even as other transactions make progress and complete.
- Timeout-based schemes, too, work without any changes in a distributed system.

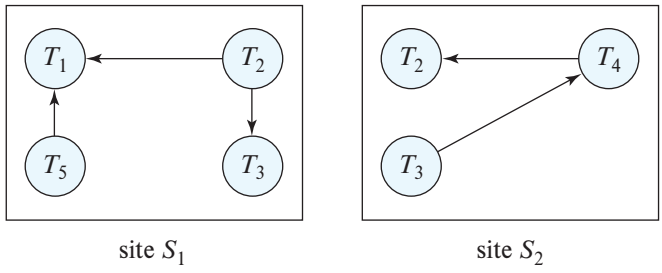

Figure 23.4 Local wait-for graphs.

Deadlock-prevention techniques may result in unnecessary waiting and rollback when used in a distributed system, just as in a centralized system,

We now consider deadlock-detection techniques that allow deadlocks to occur and detect them if they do. The main problem in a distributed system is deciding how to maintain the wait-for graph. Common techniques for dealing with this issue require that each node keep a **local wait-for graph**. The nodes of the graph correspond to all the transactions (local as well as nonlocal) that are currently either holding or requesting any of the items local to that node. For example, Figure 23.4 depicts a system consisting of two nodes, each maintaining its local wait-for graph. Note that transactions T_2 and T_3 appear in both graphs, indicating that the transactions have requested items at both nodes.

These local wait-for graphs are constructed in the usual manner for local transactions and data items. When a transaction T_i on node N_1 needs a resource in node N_2, it sends a request message to node N_2. If the resource is held by transaction T_j, the system inserts an edge $T_i \rightarrow T_j$ in the local wait-for graph of node N_2.

If any local wait-for graph has a cycle, a deadlock has occurred. On the other hand, the fact that there are no cycles in any of the local wait-for graphs does not mean that there are no deadlocks. To illustrate this problem, we consider the local wait-for graphs of Figure 23.4. Each wait-for graph is acyclic; nevertheless, a deadlock exists in the system because the *union* of the local wait-for graphs contains a cycle. This graph appears in Figure 23.5.

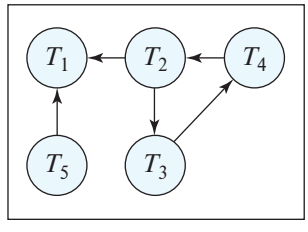

Figure 23.5 Global wait-for graph for Figure 23.4.

In the **centralized deadlock detection** approach, the system constructs and maintains a **global wait-for graph** (the union of all the local graphs) in a *single* node: the deadlock-detection coordinator. Since there is communication delay in the system, we must distinguish between two types of wait-for graphs. The *real* graph describes the real but unknown state of the system at any instance in time, as would be seen by an omniscient observer. The *constructed* graph is an approximation generated by the controller during the execution of the controller's algorithm. Obviously, the controller must generate the constructed graph in such a way that, whenever the detection algorithm is invoked, the reported results are correct. *Correct* means in this case that, if a deadlock exists, it is reported promptly, and if the system reports a deadlock, it is indeed in a deadlock state.

The global wait-for graph can be reconstructed or updated under these conditions:

- Whenever a new edge is inserted in or removed from one of the local wait-for graphs.
- Periodically, when a number of changes have occurred in a local wait-for graph.
- Whenever the coordinator needs to invoke the cycle-detection algorithm.

When the coordinator invokes the deadlock-detection algorithm, it searches its global graph. If it finds a cycle, it selects a victim to be rolled back. The coordinator must notify all the nodes that a particular transaction has been selected as the victim. The nodes, in turn, roll back the victim transaction.

This scheme may produce unnecessary rollbacks if:

- **False cycles** exist in the global wait-for graph. As an illustration, consider a snapshot of the system represented by the local wait-for graphs of Figure 23.6. Suppose that T_2 releases the resource that it is holding in node N_1, resulting in the deletion of the edge $T_1 \rightarrow T_2$ in N_1. Transaction T_2 then requests a resource held by T_3 at node N_2, resulting in the addition of the edge $T_2 \rightarrow T_3$ in N_2. If the insert $T_2 \rightarrow T_3$ message from N_2 arrives before the remove $T_1 \rightarrow T_2$ message from N_1, the coordinator may discover the false cycle $T_1 \rightarrow T_2 \rightarrow T_3$ after the insert (but before the remove). Deadlock recovery may be initiated, although no deadlock has occurred.

  Note that the false-cycle situation could not occur under two-phase locking. The likelihood of false cycles is usually sufficiently low that they do not cause a serious performance problem.

- A *deadlock* has indeed occurred and a victim has been picked, while one of the transactions was aborted for reasons unrelated to the deadlock. For example, suppose that node N_1 in Figure 23.4 decides to abort T_2. At the same time, the coordinator has discovered a cycle and has picked T_3 as a victim. Both T_2 and T_3 are now rolled back, although only T_2 needed to be rolled back.

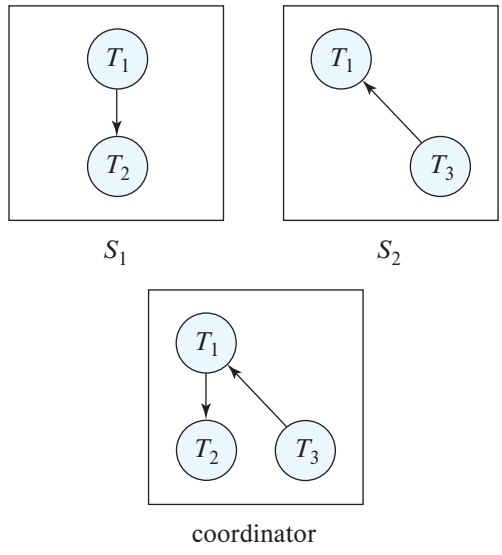

Figure 23.6 False cycles in the global wait-for graph.

Deadlock detection can be done in a distributed manner, with several nodes taking on parts of the task, instead of it being done at a single node. However, such algorithms are more complicated and more expensive. See the bibliographical notes for references to such algorithms.

23.3.3 Leases

One of the issues with using locking in a distributed system is that a node holding a lock may fail, and not release the lock. The locked data item could thus become (logically) inaccessible, until the failed node recovers and releases the lock, or the lock is released by another node on behalf of the failed node.

If an exclusive lock has been obtained on a data item, and the transaction is in the prepared state, the lock cannot be released until a commit/abort decision is made for the transaction. However, in many other cases it is acceptable for a lock that has been granted earlier to be revoked subsequently. In such cases, the concept of a lease can be very useful.

A **lease** is a lock that is granted for a specific period of time. If the process that acquires a lease needs to continue holding the lock beyond the specified period, it can **renew** the lease. A lease renewal request is sent to the lock manager, which extends the lease and responds with an acknowledgment as long as the renewal request comes in time. However, if the time expires, and the process does not renew the lease, the lease is said to **expire**, and the lock is released. Thus, any lease acquired by a node that either fails, or gets disconnected from the lock manager, is automatically released when the

lease expires. The node that holds a lease regularly compares the current lease expiry time with its local clock to determine if it still has the lease or the lease has expired.

One of the uses of leases is to ensure that there is only one coordinator for a protocol in a distributed system. A node that wants to act as coordinator requests an exclusive lease on a data item associated with the protocol. If it gets the lease, it can act as coordinator until the lease expires; as long as it is active, it requests lease renewal before the lease expires, and it continues to be the coordinator as long as the lock manager permits the lease renewal.

If a node N_1 acting as coordinator dies after the expiry of the lease period, the lease automatically expires, and another node N_2 that requests the lease can acquire it and become the coordinator. In most protocols it is important that there should be only one coordinator at a given time. The lease mechanism guarantees this, as long as clocks are synchronized. However, if the coordinator's clock runs slower than the lock manager's clock, a situation can arise where the coordinator thinks it still has the lease, while the lock manager thinks the lease has expired. While clocks cannot be exactly synchronized, in practice the inaccuracy is not very high. The lock manager waits for some extra wait time after the lease expiry time to account for clock inaccuracies before it actually treats the lease as expired.

A node that checks the local clock and decides it still has a lease may then take a subsequent action as coordinator. It is possible that the lease may have expired between when the clock was checked and when the subsequent action took place, which could result in the action taking place after the node is no longer the coordinator. Further, even if the action took place while the node had a valid lease, a message sent by the node may be delivered after a delay, by which time the node may have lost its lease. While it is possible for the network to deliver a message arbitrarily late, the system can decide on a maximum message delay time, and any message that is older is ignored by the recipient; messages have timestamps set by the sender, which are used to detect if a message needs to be ignored.

The time gaps due to the above two issues can be taken into account by checking that the lease expiry is at least some time t' into the future before initiating an action, where t' is a bound on how long the action will take after the lease time check, including the maximum message delay.

We have assumed here that while coordinators may fail, the lock manager that issues leases is able to tolerate faults. We study in Section 23.8.4 how to build a fault-tolerant lock manager; we note that the techniques described in that section are general purpose and can be used to implement fault-tolerant versions of any deterministic process, modeled as a "state machine."

23.3.4 Distributed Timestamp-Based Protocols

The principal idea behind the timestamp-based concurrency control protocols in Section 18.5 is that each transaction is given a *unique* timestamp that the system uses in

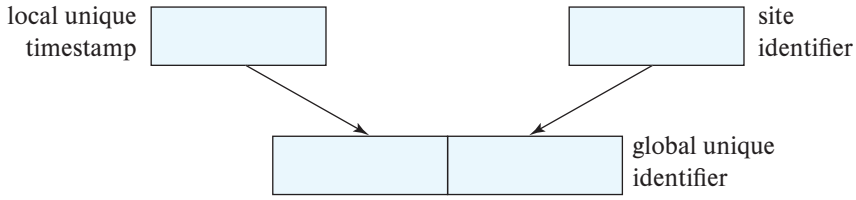

Figure 23.7 Generation of unique timestamps.

deciding the serialization order. Our first task, then, in generalizing the centralized scheme to a distributed scheme is to develop a scheme for generating unique timestamps. We then discuss how the timestamp-based protocols can be used in a distributed setting.

23.3.5 Generation of Timestamps

There are two primary methods for generating unique timestamps, one centralized and one distributed. In the centralized scheme, a single node distributes the timestamps. The node can use a logical counter or its own local clock for this purpose. While this scheme is easy to implement, failure of the node would potentially block all transaction processing in the system.

In the distributed scheme, each node generates a unique local timestamp by using either a logical counter or the local clock. We obtain the unique global timestamp by concatenating the unique local timestamp with the node identifier, which also must be unique (Figure 23.7). If a node has multiple threads running on it (as is almost always the case today), a thread identifier is concatenated with the node identifier, to make the timestamp unique. Further, we assume that consecutive calls to get the local timestamp within a node/thread will return different timestamps; if this is not guaranteed by the local clock, the returned local timestamp value may need to be incremented, to ensure two calls do not get the same local timestamp.

The order of concatenation is important! We use the node identifier in the least significant position to ensure that the global timestamps generated in one node are not always greater than those generated in another node.

We may still have a problem if one node generates local timestamps at a rate faster than that of the other nodes. In such a case, the fast node's logical counter will be larger than that of other nodes. Therefore, all timestamps generated by the fast node will be larger than those generated by other nodes. What we need is a mechanism to ensure that local timestamps are generated fairly across the system. There are two solution approaches for this problem.

1. Keep the clocks synchronized by using a network time protocol, which is a standard feature in computers today. The protocol periodically communicates with a

server to find the current time. If the local time is ahead of the time returned by the server, the local clock is slowed down, whereas if the local time is behind the time returned by the server it is speeded up, to bring it back in synchronization with the time at the server. Since all nodes are approximately synchronized with the server, they are also approximately synchronized with each other.

2. We define within each node N_i a **logical clock** (LC_i), which generates the unique local timestamp. The logical clock can be implemented as a counter that is incremented after a new local timestamp is generated. To ensure that the various logical clocks are synchronized, we require that a node N_i advance its logical clock whenever a transaction T_i with timestamp $< x, y >$ visits that node and x is greater than the current value of LC_i. In this case, node N_i advances its logical clock to the value $x + 1$. As long as messages are exchanged regularly, the logical clocks will be approximately synchronized.

23.3.6 Distributed Timestamp Ordering

The timestamp ordering protocol can be easily extended to a parallel or distributed database setting. Each transaction is assigned a globally unique timestamp at the node where it originates. Requests sent to other nodes include the transaction timestamp. Each node keeps track of the read and write timestamps of the data items at that node. Whenever an operation is received by a node, it does the timestamp checks that we saw in Section 18.5.2, locally, without any need to communicate with other nodes.

Timestamps must be reasonably synchronized across nodes; otherwise, the following problem can occur. Suppose one node has a time significantly lagging the others, and a transaction T_1 gets its timestamp at that node n_1. Suppose the transaction T_1 fails a timestamp test on a data item d_i because d_i has been updated by a transaction T_2 with a higher timestamp; T_1 would be restarted with a new timestamp, but if the time at node n_1 is not synchronized, the new timestamp may still be old enough to cause the timestamp test to fail, and T_1 would be restarted repeatedly until the time at n_1 advances ahead of the timestamp of T_2.

Note that as in the centralized case, if a transaction T_i reads an uncommitted value written by another transaction T_j, T_i cannot commit until T_j commits. This can be ensured either by making reads wait for uncommitted writes to be committed, which can be implemented using locking, or by introducing commit dependencies, as discussed in Section 18.5. The waiting time can be exacerbated by the time required to perform 2PC, if the transaction performs updates at more than one node. While a transaction T_i is in the prepared state, its writes are not committed, so any transaction with a higher timestamp that reads an item written by T_i would be forced to wait.

We also note that the *multiversion timestamp ordering protocol* can be used locally at each node, without any need to communicate with other nodes, similar to the case of the timestamp ordering protocol.

23.3.7 Distributed Validation

We now consider the validation-based protocol (also called the optimistic concurrency control protocol) that we saw in Section 18.6. The protocol is based on three timestamps:

- The start timestamp StartTS(T_i).
- The validation timestamp, TS(T_i), which is used as the serialization order.
- The finish timestamp FinishTS(T_i) which identifies when the writes of a transaction have completed.

While we saw a serial version of the validation protocol in Section 18.6, where only one transaction can perform validation at a time, there are extensions to the protocol to allow validations of multiple transactions to occur concurrently, within a single system.

We now consider how to adapt the protocol to a distributed setting.

1. Validation is done locally at each node, with timestamps assigned as described below.
2. In a distributed setting, the validation timestamp TS(T_i) can be assigned at any of the nodes, but the same timestamp TS(T_i) must be used at all nodes where validation is to be performed. Transactions must be serializable based on their timestamps TS(T_i).
3. The validation test for a transaction T_i looks at all transactions T_j with TS(T_j) < TS(T_i), to check if T_j either finished before T_i started, or has no conflicts with T_i. The assumption is that once a particular transaction enters the validation phase, no transaction with a lower timestamp can enter the validation phase. The assumption can be ensured in a centralized system by assigning the timestamps in a critical section, but cannot be ensured in a distributed setting.

   A key problem in the distributed setting is that a transaction T_j may enter the validation phase after a transaction T_i, but with TS(T_j) < TS(T_i). It is too late for T_i to be validated against T_j. However, this problem can be easily fixed by rolling back any transaction if, when it starts validation at a node, a transaction with a later timestamp had already started validation at that node.
4. The start and finish timestamps are used to identify transactions T_j whose writes would definitely have been seen by a transaction T_i. These timestamps must be assigned locally at each node, and must satisfy StartTS(T_i) $\leq$ TS(T_i) $\leq$ FinishTS(T_i). Each node uses these timestamps to perform validation locally.
5. When used in conjunction with 2PC, a transaction must first be validated and then enter the prepared state. Writes cannot be committed at the database until

the transaction enters the committed state in 2PC. Suppose a transaction T_j reads an item updated by a transaction T_i that is in the prepared state and is allowed to proceed using the old value of the data item (since the value generated by T_i has not yet been written to the database). Then, when transaction T_j attempts to validate, it will be serialized after T_i and will surely fail validation if T_i commits. Thus, the read by T_j may as well be held until T_i commits and finishes its writes. The above behavior is the same as what would happen with locking, with write locks acquired at the time of validation.

Although full implementations of validation-based protocols are not widely used in distributed settings, optimistic concurrency control without read validation, which we saw in Section 18.9.3, is widely used in distributed settings. Recall that the scheme depends on storing a version number with each data item, a feature that is supported by many key-value stores.[1] Version numbers are incremented each time the data item is updated.

Validation is performed at the time of writing the data item, which can be done using a test-and-set function based on version numbers, that is supported by some key-value stores. This function allows an update to a data item to be conditional on the current version of the data item being the same as a specified version number. If the current version number of the data item is more recent than the specified version number, the update is not performed. For example, a transaction that read version 7 of a data item can perform a write, conditional on the version still being at 7. If the item has been updated meanwhile, the current version would not match, and the write would fail; however, if the version number is still 7, the write would be performed successfully, and the version number incremented to 8.

The test-and-set function can thus be used by applications to implement the limited form of validation-based concurrency control, discussed in Section 18.9.3, at the level of individual data items. Thereby, a transaction could read a value from a data item, perform computation locally, and update the data item at the end, as long as the value it read has not changed subsequently. This approach does not guarantee overall serializability, but it does prevent the lost-update anomaly.

HBase supports the test-and-set operation based on comparing values (similar to the hardware test-and-set operation), which is called `checkAndPut()`. Instead of comparing to a system-generated version number, the `checkAndPut()` invocation can pass in a column and a value; the update is performed only if the row has the specified value for the specified column. The check and the update are performed atomically. A variant, `checkAndMutate()`, allows multiple modifications to a row, such as adding or updating a column, deleting a column, or incrementing a column, after checking a condition, as a single atomic operation.

[1]Note that this is not the same as multiversioning, since only one version needs to be stored.

23.4 Replication

One of the goals in using distributed databases is **high availability**; that is, the database must function almost all the time. In particular, since failures are more likely in large distributed systems, a distributed database must continue functioning even when there are various types of failures. The ability to continue functioning even during failures is referred to as **robustness**.

For a distributed system to be robust, data must be replicated, allowing the data to be accessible even if a node containing a replica of the data fails.

The database system must keep track of the locations of the replicas of each data item in the database catalog. Replication can be at the level of individual data items, in which the catalog will have one entry for each data item, recording the nodes where it is replicated. Alternatively, replication can be done at the level of partitions of a relation, with an entire partition replicated at two or more nodes. The catalog would then have one entry for each partition, resulting in considerably lower overhead than having one entry for each data item.

In this section we first discuss (in Section 23.4.1) issues with consistency of values between replicas. We then discuss (in Section 23.4.2) how to extend concurrency control techniques to deal with replicas, ignoring the issue of failures. Further extensions of the techniques to handle failures but modifying how reads and writes are executed are described in Section 23.4.3.

23.4.1 Consistency of Replicas

Given that a data item (or partition) is replicated, the system should ideally ensure that the copies have the same value. Practically, given that some nodes may be disconnected or may have failed, it is impossible to ensure that all copies have the same value. Instead, the system must ensure that even if some replicas do not have the latest value, reads of a data item get to see the latest value that was written.

More formally, the implementations of read and write operations on the replicas of a data item must follow a protocol that ensures the following property, called **linearizability**: Given a set of read and write operations on a data item,

1. there must be a linear ordering of the operations such that each read in the ordering should see the value written by the most recent write preceding the read (or the initial value if there is no such write), and
2. if an operation o_1 finishes before an operation o_2 begins (based on external time), then o_1 must precede o_2 in the linear order.

Note that linearizability only addresses what happens to a single data item, and it is orthogonal to serializability.

We first consider approaches that write all copies of a data item and discuss limitations of this approach; in particular, to ensure availability during failure, failed nodes need to be removed from the set of replicas, which can be quite tricky as we will see.

It is not possible, in general, to differentiate between node failure and network partition. The system can usually detect that a failure has occurred, but it may not be able to identify the type of failure. For example, suppose that node N_1 is not able to communicate with N_2. It could be that N_2 has failed. However, another possibility is that the link between N_1 and N_2 has failed, resulting in network partition. The problem is partly addressed by using multiple links between nodes, so that even if one link fails the nodes will remain connected. However, multiple link failures can still occur, so there are situations where we cannot be sure whether a node failure or network partition has occurred.

There are protocols for data access that can continue working even if some nodes have failed, without any explicit actions to deal with the failures, as we shall see in Section 23.4.3.1. These protocols are based on ensuring a majority of nodes are written/read. With such protocols, actions to detect failed nodes and remove them from the system can be done in the background, and (re)integration of new or recovered nodes into the system can also be done without disrupting processing.

Although traditional database systems place a premium on consistency, there are many applications today that value availability more than consistency. The design of replication protocols is different for such systems and is discussed in Section 23.6.

In particular, one such alternative that is widely used for maintaining replicated data is to perform the update on a primary copy of the data item, and allow the transaction to commit without updating the other copies. However, the update is subsequently propagated to the other copies. Such propagation of updates, referred to as *asynchronous replication* or *lazy propagation of updates*, is discussed in Section 23.6.2.

One drawback of asynchronous replication is that replicas may be out of date for some time following each update. Another drawback is that if the primary copy fails after a transaction commits, but before the updates were propagated to the replicas, the updates of the committed transaction may not be visible to subsequent transactions, leading to an inconsistency.

On the other hand, a major benefit of asynchronous replication is that exclusive locks can be released as soon as the transaction commits on the primary copy. In contrast, if other replicas have to be updated before the transaction commits, there may be a significant delay in committing the transaction. In particular, if data is geographically replicated to ensure availability despite failure of an entire data center, the network round-trip time to a remote data center could range from tens of milliseconds to nearby locations, up to hundreds of milliseconds for data centers that are on the other side of the world. If a transaction were to hold a lock on a data item for this duration, the number of transactions that can update that data item would be limited to approximately 10 to 100 transactions per second. For certain applications, for example, user data in a web application, 10 to 100 transactions per second for a single data item is quite sufficient. However, for applications where some data items are updated

by a large number of transactions each second, holding locks for such a long time is not acceptable. Asynchronous replication may be preferred in such cases.

23.4.2 Concurrency Control with Replicas

We discuss several alternative ways of dealing with locking in the presence of replication of data items, in Section 23.4.2.1 to Section 23.4.2.4.

In this section, we assume updates are done on all replicas of a data item. If any node containing a replica of a data item has failed, or is disconnected from the other nodes, that replica cannot be updated. We discuss how to perform reads and updates in the presence of failures later, in Section 23.4.3.

23.4.2.1 Primary Copy

When a system uses data replication, we can choose one of the replicas of a data item as the **primary copy**. For each data item Q, the primary copy of Q must reside in precisely one node, which we call the **primary node** of Q.

When a transaction needs to lock a data item Q, it requests a lock at the primary node of Q. As before, the response to the request is delayed until it can be granted. The primary copy enables concurrency control for replicated data to be handled like that for unreplicated data. This similarity allows for a simple implementation. However, if the primary node of Q fails, lock information for Q would be lost, and Q would be inaccessible, even though other nodes containing a replica may be accessible.

23.4.2.2 Majority Protocol

The **majority protocol** works this way: If data item Q is replicated in n different nodes, then a lock-request message must be sent to more than one-half of the n nodes in which Q is stored. Each lock manager determines whether the lock can be granted immediately (as far as it is concerned). As before, the response is delayed until the request can be granted. The transaction does not operate on Q until it has successfully obtained a lock on a majority of the replicas of Q.

We assume for now that writes are performed on all replicas, requiring all nodes containing replicas to be available. However, the major benefit of the majority protocol is that it can be extended to deal with node failures, as we shall see in Section 23.4.3.1. The protocol also deals with replicated data in a decentralized manner, thus avoiding the drawbacks of central control. However, it suffers from these disadvantages:

- **Implementation**. The majority protocol is more complicated to implement than are the previous schemes. It requires at least $2(n/2 + 1)$ messages for handling lock requests and at least $(n/2 + 1)$ messages for handling unlock requests.
- **Deadlock handling**. In addition to the problem of global deadlocks due to the use of a distributed-lock-manager approach, it is possible for a deadlock to occur even if only one data item is being locked. As an illustration, consider a system with four nodes and full replication. Suppose that transactions T_1 and T_2 wish to lock

data item Q in exclusive mode. Transaction T_1 may succeed in locking Q at nodes N_1 and N_3, while transaction T_2 may succeed in locking Q at nodes N_2 and N_4. Each then must wait to acquire the third lock; hence, a deadlock has occurred. Luckily, we can avoid such deadlocks with relative ease by requiring all nodes to request locks on the replicas of a data item in the same predetermined order.

23.4.2.3 Biased Protocol

The **biased protocol** is another approach to handling replication. The difference from the majority protocol is that requests for shared locks are given more favorable treatment than requests for exclusive locks.

- **Shared locks**. When a transaction needs to lock data item Q, it simply requests a lock on Q from the lock manager at one node that contains a replica of Q.
- **Exclusive locks**. When a transaction needs to lock data item Q, it requests a lock on Q from the lock manager at all nodes that contain a replica of Q.

As before, the response to the request is delayed until it can be granted.

The biased scheme has the advantage of imposing less overhead on `read` operations than does the majority protocol. This savings is especially significant in common cases in which the frequency of `read` is much greater than the frequency of `write`. However, the additional overhead on writes is a disadvantage. Furthermore, the biased protocol shares the majority protocol's disadvantage of complexity in handling deadlock.

23.4.2.4 Quorum Consensus Protocol

The **quorum consensus** protocol is a generalization of the majority protocol. The quorum consensus protocol assigns each node a nonnegative weight. It assigns read and write operations on an item x two integers, called **read quorum** Q_r and **write quorum** Q_w, that must satisfy the following condition, where S is the total weight of all nodes at which x resides:

$$Q_r + Q_w > S \textbf{ and } 2 * Q_w > S$$

To execute a read operation, enough replicas must be locked that their total weight is at least Q_r. To execute a write operation, enough replicas must be locked so that their total weight is at least Q_w.

A benefit of the quorum consensus approach is that it can permit the cost of either read or write locking to be selectively reduced by appropriately defining the read and write quorums. For instance, with a small read quorum, reads need to obtain fewer locks, but the write quorum will be higher, hence writes need to obtain more locks. Also, if higher weights are given to some nodes (e.g., those less likely to fail), fewer

nodes need to be accessed for acquiring locks. In fact, by setting weights and quorums appropriately, the quorum consensus protocol can simulate the majority protocol and the biased protocols.

Like the majority protocol, quorum consensus can be extended to work even in the presence of node failures, as we shall see in Section 23.4.3.1.

23.4.3 Dealing with Failures

Consider the following protocol to deal with replicated data. Writes must be successfully performed at all replicas of a data item. Reads may read from any replica. When coupled with two-phase locking, such a protocol will ensure that reads will see the value written by the most recent write to the same data item. This protocol is also called the **read one, write all copies** protocol since all replicas must be written, and any replica can be read.

The problem with this protocol lies in what to do if some node is unavailable. To allow work to proceed in the event of failures, it may appear that we can use a "read one, write all available" protocol. In this approach, a read operation proceeds as in the **read one, write all** scheme; any available replica can be read, and a read lock is obtained at that replica. A write operation is shipped to all replicas, and write locks are acquired on all the replicas. If a node is down, the transaction manager proceeds without waiting for the node to recover. While this approach appears very attractive, it does not guarantee consistency of writes and reads. For example, a temporary communication failure may cause a node to appear to be unavailable, resulting in a write not being performed, but when the link is restored, the node is not aware that it has to perform some reintegration actions to catch up on writes it has lost. Further, if the network partitions, each partition may proceed to update the same data item, believing that nodes in the other partitions are all dead.

23.4.3.1 Robustness Using the Majority-Based Protocol

The majority-based approach to distributed concurrency control in Section 23.4.2.2 can be modified to work in spite of failures. In this approach, each data object stores with it a version number to detect when it was last written. Whenever a transaction writes an object it also updates the version number in this way:

- If data object a is replicated in n different nodes, then a lock-request message must be sent to more than one-half of the n nodes at which a is stored. The transaction does not operate on a until it has successfully obtained a lock on a majority of the replicas of a.

  Updates to the replicas can be committed atomically using 2PC. (We assume for now that all replicas that were accessible stay accessible until commit, but we

relax this requirement later in this section, where we also discuss alternatives to 2PC.)

- Read operations look at all replicas on which a lock has been obtained and read the value from the replica that has the highest version number. (Optionally, they may also write this value back to replicas with lower version numbers.) Writes read all the replicas just like reads to find the highest version number (this step would normally have been performed earlier in the transaction by a read, and the result can be reused). The new version number is one more than the highest version number. The write operation writes all the replicas on which it has obtained locks and sets the version number at all the replicas to the new version number.

Failures (whether network partitions or node failures) can be tolerated as long as (1) the nodes available at commit contain a majority of replicas of all the objects written to and (2) during reads, a majority of replicas are read to find the version numbers. If these requirements are violated, the transaction must be aborted. As long as the requirements are satisfied, the two-phase commit protocol can be used, as usual, on the nodes that are available.

In this scheme, reintegration is trivial; nothing needs to be done. This is because writes would have updated a majority of the replicas, while reads will read a majority of the replicas and find at least one replica that has the latest version.

However, the majority protocol using version numbers has some limitations, which can be avoided by using extensions or by using alternative protocols.

1. The first problem is how to deal with the failure of participants during an execution of the two-phase commit protocol.

   This problem can be dealt with by an extension of the two-phase commit protocol that allows commit to happen even if some replicas are unavailable, as long as a majority of replicas of a partition confirm that they are in prepared state. When participants recover or get reconnected, or otherwise discover that they do not have the latest updates, they need to query other nodes to catch up on missing updates. References that provide details of such solutions may be found in the bibliographic notes for this chapter, available online.

2. The second problem is how to deal with the failure of the coordinator during an execution of two-phase commit protocol, which could lead to the blocking problem. Consensus protocols, which we study in Section 23.8, provide a robust way of implementing two-phase commit without the risk of blocking even if the coordinator fails, as long as a majority of the nodes are up and connected, as we will see in Section 23.8.5.

3. The third problem is that reads pay a higher price, having to contact a majority of the copies. We study approaches to reducing the read overhead in Section 23.4.3.2.

23.4.3.2 Reducing Read Cost

One approach to dealing with this problem is to use the idea of read and write quorums from the quorum consensus protocol; reads can read from a smaller read quorum, while writes have to successfully write to a larger write quorum. There is no change to the version numbering technique described earlier. The drawback of this approach is that a higher write quorum increases the chance of blocking of update transactions, due to failure or disconnection of nodes. As a special case of quorum consensus, we give unit weights to all nodes, set the read quorum to 1, and set the write quorum to n (all nodes). This corresponds to the read-any-write-all protocol we saw earlier. There is no need to use version numbers with this protocol. However, if even a single node containing a data item fails, no write to the item can proceed, since the write quorum will not be available.

A second approach is to use the primary copy technique for concurrency control and force all updates to go through the primary copy. Reads can be satisfied by accessing only one node, in contrast to the majority or quorum protocols. However, an issue with this approach is how to handle failures. If the primary copy node fails, and another node is assigned to act as the primary copy, it must ensure that it has the latest version of all data items. Subsequently, reads can be done at the primary copy, without having to read data from other copies.

This approach requires that there be at most one node that can act as primary copy at a time, even in the event of network partitions. This can be ensured using leases as we saw earlier in Section 23.7. Furthermore, this approach requires an efficient way for the new coordinator to ensure that it has the latest version of all data items. This can be done by having a log at each node and ensuring the logs are consistent with each other. This problem is by itself a nontrivial process, but it can be solved using distributed consensus protocols which we study in Section 23.8. Distributed consensus internally uses a majority scheme to ensure consistency of the logs. But it turns out that if distributed consensus is used to keep logs synchronized, there is no need for version numbering.

In fact, consensus protocols provide a way of implementing fault-tolerant replication of data, as we see later in Section 23.8.4. Many fault-tolerant storage system implementations today are built using fault-tolerant replication of data based on consensus protocols.

There is a variant of the primary copy scheme, called the **chain replication** protocol, where the replicas are ordered. Each update is sent to the first replica, which records it locally and forwards it to the next replica, and so on. The update is completed when the last (tail) replica receives the update. Reads must be executed at the tail replica, to ensure that only updates that have been fully replicated are read. If a node in a replica chain fails, reconfiguration is required to update the chain; further, the system must ensure that any incomplete updates are completed before processing further updates. Optimized versions of the chain replication scheme are used in several storage systems.

References providing more details of the chain replication protocol may be found in the Further Reading section at the end of the chapter.

23.4.4 Reconfiguration and Reintegration

While nodes do fail, in most cases nodes recover soon, and the protocols described earlier can ensure that they will catch up with any updates that they missed.

However, in some cases a node may fail permanently. The system must then be **reconfigured** to remove failed nodes, and to allow other nodes to take over the tasks assigned to the failed node. Further, the database catalog must be updated to remove the failed node from the list of replicas of all data items (or relation partitions) that were replicated at that node.

As discussed earlier, a network failure may result in a node appearing to have failed, even if it has not actually failed. It is safe to remove such a node from the list of replicas; reads will no longer be routed to the node even though it may be accessible, but that will not cause any consistency problems.

If a failed node that was removed from the system eventually recovers, it must be **reintegrated** into the system. When a failed node recovers, if it had replicas of any partition or data item, it must obtain the current values of these data items it stores. The database recovery log at a live site can be used to find and perform all updates that happened when the node was down,

Reintegration of a node is more complicated than it may seem to be at first glance, since there may be updates to the data items processed during the time that the node is recovering. The database recovery log at a live site is used for catching up with the latest values for all data items at the node. Once it has caught up with the current value of all data items, the node should be added back into the list of replicas for the relevant partitions/data items, so it will receive all future updates. Locks are obtained on the partitions/data items, updates up to that point are applied from the log, and the node is added to the list of replicas for the partitions or data items, before releasing the locks. Subsequent updates will be applied directly to the node, since it will be in the list of replicas.

Reintegration is much easier with the majority-based protocols in Section 23.4.3.1, since the protocol can tolerate nodes with out-of-date data. In this case, a node can be reintegrated even before catching up on updates, and the node can catch up with missed updates subsequently.

Reconfiguration depends on nodes having an up-to-date version of the catalog that records what table partitions (or data items) are replicated at what nodes; thus information must be consistent across all nodes in a system. The replication information in the catalog could be stored centrally, and consulted on each access, but such a design would not be scalable since the central node would be consulted very frequently and would get overloaded. To avoid such a bottleneck, the catalog itself needs to be partitioned, and it may be replicated, for example, using the majority protocol.